

# **CPT202 Assignment 2**

## **Group Report for Software Engineering Group Project**

2024/2025 Semester 2  
An online meeting booking system  
27 April 2025  
<http://121.43.36.102/>

Group number: 17

Submitted by: < Xiangyi Zhu, 2251651 >

Group Members:

1. < Xiangyi Zhu, 2251651 >
2. < Xinrong Shi, 2251997 >
3. < Zimo Zheng, 2147369 >
4. < Nan Lin, 2252173 >
5. < Xiaoya Li, 2255207 >
6. < Ziwei Zhou, 2253073 >
7. < Jiayi Wang, 2255986 >
8. < Hanqiu Zhang, 2251903 >
9. < Xinyuan Zhang, 2255403 >

## Contents

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>1. Introduction .....</b>                                 | <b>3</b>  |
| 1.1. Background and Requirements .....                       | 3         |
| 1.2. Aims of The Project .....                               | 3         |
| 1.3. User Characteristics .....                              | 3         |
| 1.4. Project Scope .....                                     | 4         |
| 1.5. Assumptions and Dependencies .....                      | 4         |
| 1.6. Project Risks .....                                     | 4         |
| <b>2. Architectural Design .....</b>                         | <b>4</b>  |
| 2.1. System Architecture .....                               | 4         |
| 2.2. Frontend System .....                                   | 5         |
| 2.3. Backend System .....                                    | 6         |
| 2.4. High-level Database Design .....                        | 7         |
| <b>3. Software Design .....</b>                              | <b>7</b>  |
| 3.1. Software Modules .....                                  | 7         |
| 3.2. High-level Process Flow .....                           | 8         |
| 3.3. Software Support Services .....                         | 10        |
| 3.4. Coding Structure and Convention .....                   | 11        |
| 3.5. Software Configuration and Production Environment ..... | 12        |
| <b>4. Software testing .....</b>                             | <b>12</b> |
| 4.1. Unit Test .....                                         | 12        |
| 4.2. Integration Test .....                                  | 13        |
| 4.3. Acceptance Test .....                                   | 13        |
| <b>5. Conclusion and future work .....</b>                   | <b>14</b> |
| <b>6. References .....</b>                                   | <b>14</b> |
| <b>7. Appendix .....</b>                                     | <b>17</b> |

# 1. Introduction

## 1.1. Background and Requirements

In recent years, the increasing complexity of academic and administrative activities within schools has led to a significant increase in the demand for online conference room booking systems. This trend highlights the urgent need for efficient resource management and convenient collaboration tools. However, many schools' existing conference room booking systems still face many challenges, including complex user interfaces, inefficient booking processes, insufficient conflict management, and lack of real-time notification functions. These issues often lead to poor user experience, low resource utilization, and inefficient communication (Sanchez *et al.*, 2019).

In order to effectively address these challenges, it is essential to develop a robust and user-friendly school conference room booking system. The system should be designed with users in mind, provide an intuitive interface and simplify the booking process, support real-time resource status updates to prevent conflicts, and integrate automatic notification functions to ensure that users receive booking confirmations and reminders in a timely manner. In addition, the system should also be seamlessly connected with existing school scheduling, payment systems, and permission management tools to comprehensively improve administrative efficiency (Shili *et al.*, 2009).

By optimizing resource allocation and improving user experience, the system can not only help schools manage conference room resources more efficiently, but also provide teachers and students with a smoother collaborative environment, thereby promoting the orderly development of academic and administrative activities.

## 1.2. Aims of The Project

The primary objective of this project is to design and implement an intuitive web-based school meeting room booking system:

- **For users** (e.g., students and teachers), they can register for an account to become platform users. The system provides a simple and user-friendly interface for viewing the availability of meeting rooms, submitting booking requests, and receiving real-time notifications and confirmations.
- **For administrators**, the system offers a management backend for monitoring and managing meeting room resources. This backend supports a range of functions, including real-time updates on meeting room status, automatic detection of booking conflicts, user permission management, and data visualization analysis. These features help administrators allocate resources more efficiently, optimize meeting room usage, and support the school's teaching and administrative activities.

## 1.3. User Characteristics

The system supports three main user roles: ordinary users, administrators, and super administrators.

Ordinary users can log in to the system to view conference rooms, submit reservation applications, view reservation history, cancel reservations, submit maintenance requests, and view notifications. Administrators (usually department-level or floor managers) are responsible for approving reservation applications, monitoring conference room usage, handling maintenance requests, managing conference room status, and issuing internal notifications.

As the highest-level operator of the system, the super administrator is responsible for maintaining data integrity and system performance. His main permissions include managing user roles, configuring conference room classifications, adding, updating or deactivating conference rooms, maintaining conference room information, managing permission allocation, and reviewing system logs.

The three roles work together to ensure the efficient operation of the system and achieve the orderly advancement of resource optimization and administrative supervision.

## 1.4. Project Scope

The scope of the project is illustrated in the Figure 1:

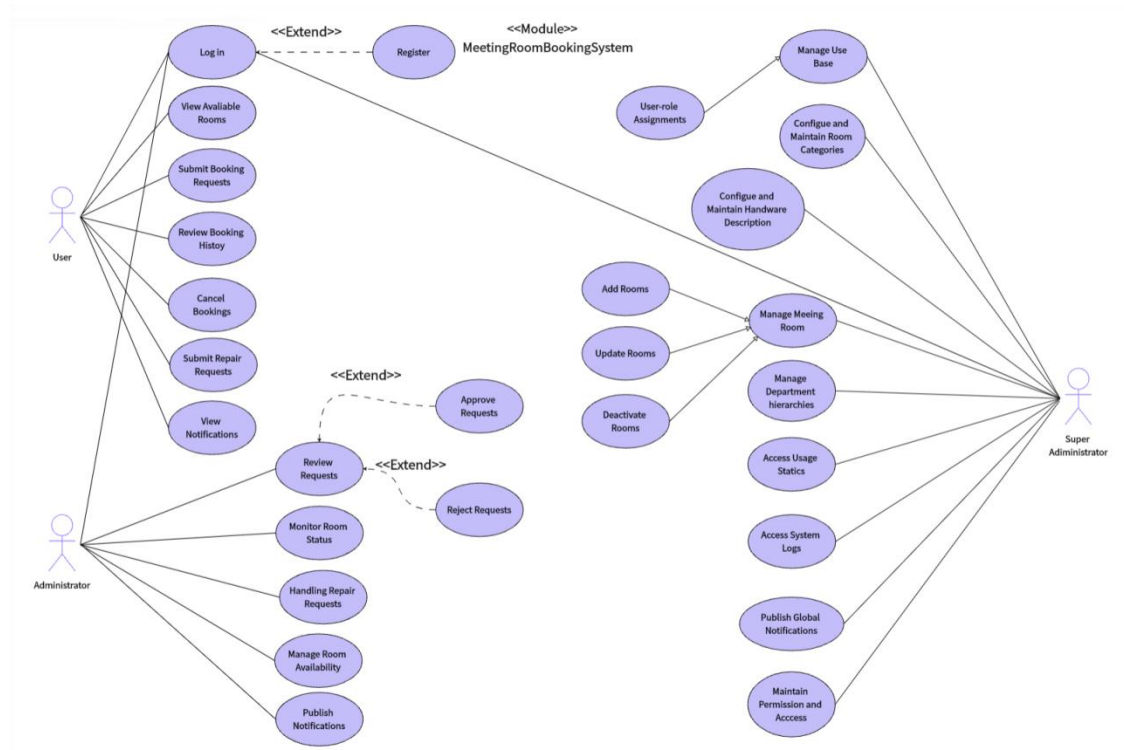


Figure 1: the Use Case Diagram of the System

## 1.5. Assumptions and Dependencies

The successful implementation of the project depends on a series of prerequisites and external environmental factors, such as resource availability, technical support, team collaboration, and data integrity. In addition, the project also depends on the stability of third-party API services (e.g., email notifications), reliable network connections, and the fulfillment of external service agreements. During the project implementation process, these prerequisites and dependencies need to be continuously monitored and managed to reduce potential risks and ensure the smooth achievement of project goals.

## 1.6. Project Risks

Although this project has brought many opportunities in terms of conference room booking management and improving team reporting and presentation efficiency, there are also some risks that need to be taken seriously during the implementation process. These risks include uneven resource allocation, technical challenges such as system integration and data processing, potential security risks, and rigorous testing that must be carried out to ensure system stability and performance. In addition, the smooth implementation of the project also depends on good collaboration between teams, sufficient supply of resources, and stable support from external services. Throughout the project process, these factors need to be continuously paid attention to and managed to effectively reduce risks and ensure the smooth realization of project goals.

# 2. Architectural Design

## 2.1. System Architecture

Our Meeting Booking System employs the MVC (Model-View-Controller) architecture due to its clear separation of concerns, modularity, and scalability, making it suitable for dynamic interactions, secure data management, and multi-role functionality (Ganeshan, 2016).

### Architectural Justification :

MVC separates the system into distinct layers—the Model managing data and logic, the View presenting user interfaces, and the Controller coordinating interactions between them. This structure significantly improves maintainability, testability, and promotes effective collaborative development within a Scrum framework.

### Implementation in Our Project :

- **Model Layer:**

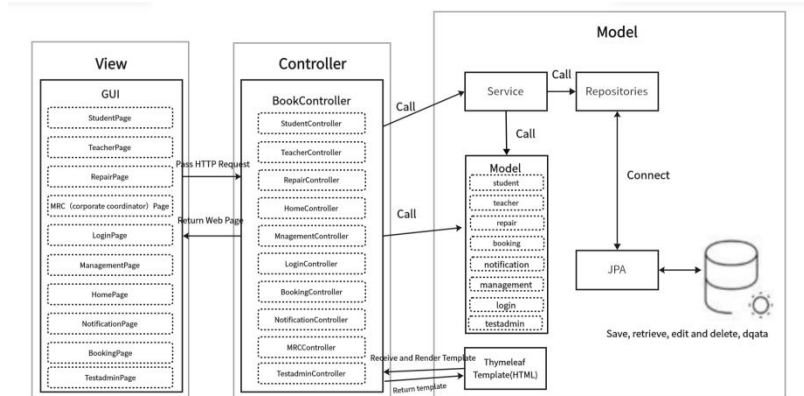
Our Model layer uses MySQL with MyBatis-Plus and MyBatis-Plus-Join as ORM frameworks. Tables follow third normal form (3NF) with foreign keys for integrity among key entities. Druid provides efficient database connection pooling and monitoring.

- **View Layer:**

Built with Vue.js 3 (Shavin, 2023) and TypeScript for type safety, our frontend leverages Vite for rapid builds, Vue Router for routing, and Element Plus for UI components. Axios manages RESTful API calls, while ECharts and FullCalendar support data visualization and scheduling. Modular component structures promote reusability and maintainability.

- **Controller Layer:**

Spring Boot facilitates RESTful API development, handling HTTP requests and orchestrating business logic across services and mappers (Heckler, 2021). Sa-Token manages authentication and role-based access, with password encryption secured by Spring Security Crypto.



**Figure 2: MVC architecture**

Although the current system does not integrate message queues or caching servers, the modular design ensures future scalability, allowing such components to be introduced if needed to optimize asynchronous processing and enhance data retrieval performance.

In summary, by leveraging the MVC architecture along with a robust technology stack and structured development workflow, we have built a secure, modular, and scalable Meeting Booking System capable of supporting the evolving needs of institutional meeting room management.

## 2.2. Frontend System

The frontend of our Meeting Booking System is developed using Vue.js 3, a progressive JavaScript framework renowned for its performance, modularity, and seamless integration with modern development toolchains. The adoption of TypeScript across the codebase enhances type safety and maintainability. The architecture follows a Single Page Application (SPA) pattern, enabling dynamic view rendering and responsive user interactions without requiring frequent page reloads.

### Core Technologies and Frameworks :

The system integrates Vite as the build tool, providing ultra-fast development and hot module replacement (HMR). Vue Router manages nested and dynamic routing across different user roles, ensuring structured navigation. The UI layer leverages Element Plus to deliver reusable and accessible components such as modals, forms, calendars,

and tables. Axios handles efficient RESTful API communication with backend services. For scheduling functionalities, FullCalendar enables intuitive drag-and-drop interfaces, while ECharts powers interactive dashboards, visualizing room usage trends through bar, line, and pie charts.

#### **UI/UX & Optimization :**

To improve user experience, Vue-Tippy is employed to provide contextual tooltips and instructional prompts. Role-based routing guards are implemented to enforce access control, restricting unauthorized views. Frontend performance is further optimized through lazy-loading routes and asynchronous imports of resource-intensive modules like ECharts and FullCalendar.

Notably, for MRC and Test Admin users, Vue's <keep-alive> component ensures that dashboard panels remain continuously active across page transitions, delivering a seamless and uninterrupted management experience.

In summary, the frontend system delivers a modular, dynamic, and maintainable user interface tailored to the diverse functional requirements of the Meeting Booking System. With strong integration of visualization, interactivity, and security, it provides a robust and scalable client-side foundation for the overall platform.

### **2.3. Backend System**

The backend of the Meeting Booking System is designed using a layered Model-View-Controller (MVC) architecture to achieve clear separation of concerns, modular scalability, and ease of maintenance.

Spring Boot serves as the core framework for developing RESTful APIs, supported by MyBatis-Plus for database operations and Sa-Token for authentication and authorization. This technology stack was selected for its maturity, active community support, and suitability for building secure, scalable enterprise-level applications.

#### **Controller Layer:**

The Controller layer, built with Spring Boot's @RestController, maps incoming HTTP requests to corresponding service operations, exposing APIs for critical functionalities such as meeting room booking management, room availability maintenance, user and department administration, repair issue tracking, and room usage statistics visualization.

Spring Security enforces request-level authentication and protects sensitive endpoints (e.g., booking approvals, user role modifications), while Sa-Token manages token-based authentication, session persistence, and fine-grained role authorization across different user types, ensuring secure and context-aware access for Students, Teachers, MRC Coordinators, Repair Staff, and Super Admins.

#### **Service Layer:**

The Service layer encapsulates the business logic of the Meeting Booking System, acting as a bridge between controllers and data persistence layers. It validates meeting room booking requests, manages approval workflows for MRC users, processes room status updates (e.g., availability, maintenance), handles user authentication and secure password management (via Sa-Token and Spring Security Crypto), generates room usage statistics for administrative dashboards, and tracks repair submissions and resolutions.

The use of @Service annotations and dependency injection supports modular business design, improves maintainability, and facilitates unit testing across key service modules.

#### **DAO (Data Access Object) Layer:**

The DAO layer abstracts database interactions using MyBatis-Plus and MyBatis-Plus-Join, simplifying CRUD operations and multi-table joins. This approach reduces boilerplate SQL code, accelerates development, and minimizes potential database inconsistencies. Key operations include managing meeting rooms, booking records, user-role mappings, and statistical queries.

By encapsulating database logic, this layer provides a reusable and centralized structure for persistence access (Li *et al.*, 2020).

#### **Entity Layer:**

The Entity layer defines Java classes mapped directly to relational tables within a MySQL database. Entities such as sys\_user, meeting\_room, meeting\_book, sys\_dept, and repair represent core business objects. Annotations like @TableName and @TableField ensure tight coupling between object models and database schema, while foreign keys and association tables maintain data integrity.

This well-defined backend architecture supports efficient request handling, secure data management, and the modular scalability required for the ongoing evolution of the Meeting Booking System.

### 2.4. High-level Database Design

Our database design utilizes MySQL to support the persistent storage and logical organization of core entities in the Meeting Booking System. The schema, normalized through an Entity-Relationship Diagram (ERD), ensures a scalable and consistent data model aligned with the system’s operational logic. Table relationships are maintained through foreign key constraints to enforce referential integrity (DuBois, 2013). This design ensures minimal redundancy, optimized query performance, and high data integrity.

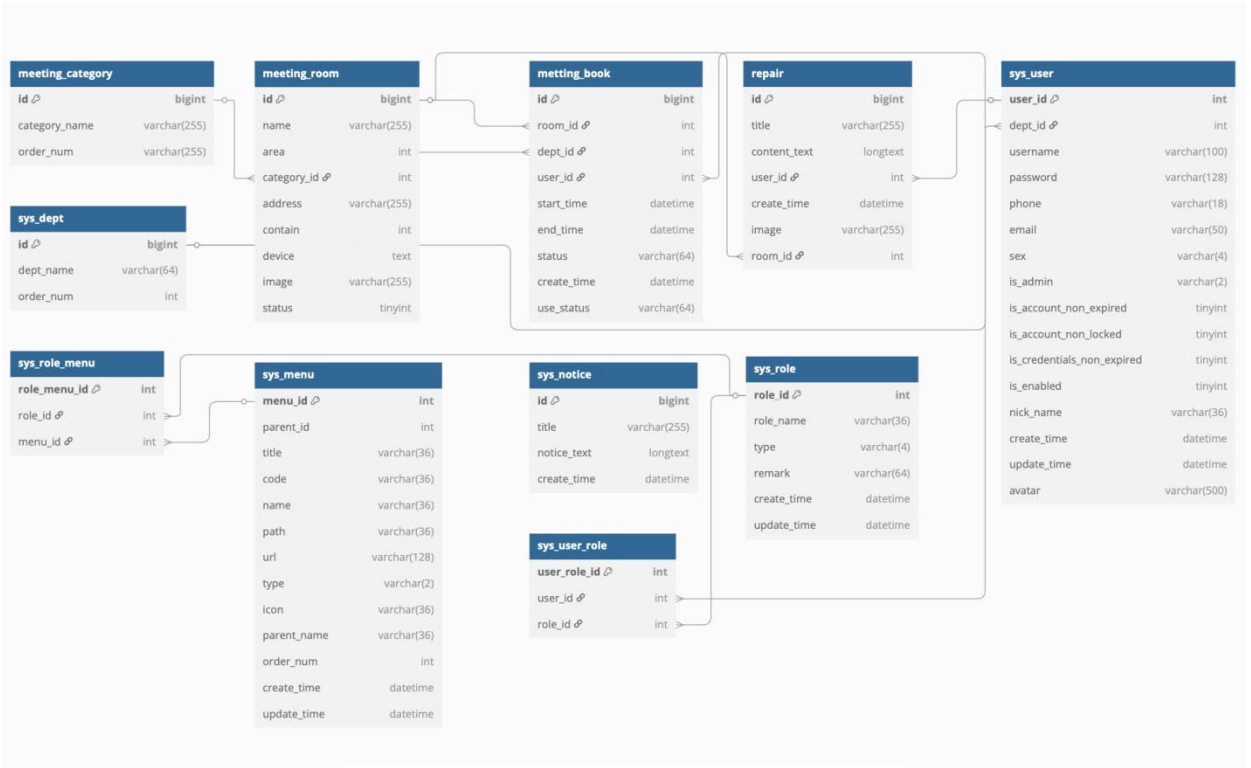


Figure 3 : ER Diagram

#### Entity Identification :

These entities form the backbone of the system, supporting multi-role interactions, scheduling processes, access management, and communication flows.

#### Data Normalization:

We followed the third normal form (3NF) principle in database design to ensure data independence, reduce redundancy, and maintain data relevance. By dividing data of different topics into independent tables and setting foreign key relationships, the system can maintain consistency when updating data, and daily maintenance is more efficient. The final entity relationship diagram (ERD) matches the modular design of the overall system, ensuring that each data table can directly correspond to the relevant service interface, while laying the foundation for subsequent function expansion and system upgrades (Mendoza and Piedra, 2020)

## 3. Software Design

### 3.1. Software Modules

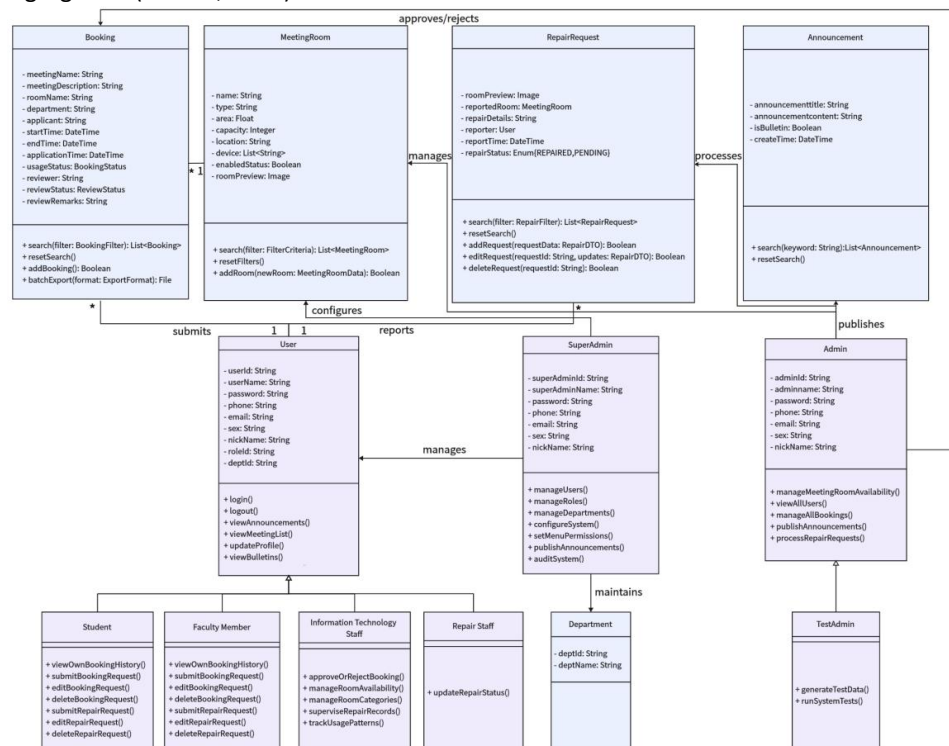
The Meeting Booking System is organized into five core software modules, each aligned with a distinct user role: Student, Teacher, MRC (Conference Coordinator), Repair Staff, and Super Admin (Test Admin). Each module grants role-specific access to functionalities tailored to the user's operational needs.

**1. Student and Teacher Module:** Students and teachers serve as standard users, able to browse available meeting rooms, submit booking requests, view booking histories, report maintenance issues, and access announcements and bulletins. Teachers may have additional privileges depending on institutional policies, but both roles primarily focus on personal meeting management.

**2. MRC (Conference Coordinator) Module:** MRC users oversee meeting room scheduling, reviewing and approving booking requests, managing room availability and categories, tracking repair records, and monitoring room usage patterns. This module ensures the orderly allocation and efficient utilization of meeting resources.

**4. Repair Staff Module:** Repair staff are responsible for processing maintenance requests. They can view submitted repair reports, update repair statuses (e.g., pending, in progress, completed), and manage supporting documentation such as images or maintenance notes through a task-focused dashboard.

**5. Super Admin (Test Admin) Module:** Super Admins possess full administrative privileges, including user, department, and role management, system configuration, menu permission settings, and announcement publishing. They are responsible for system-wide oversight, deployment, and high-level maintenance. In addition to role-specific modules, the system provides reusable core functionalities such as the Announcement and Repair Management modules, which offer consistent user experiences across roles while maintaining clear functional segregation (Farouk, 2017).



**Figure 4: Software Modules Design**

From a modular design perspective, each module exhibits high cohesion — with related business logic tightly grouped - and minimal interdependencies, ensuring scalability, maintainability, and ease of future expansion.

### 3.2. High-level Process Flow

The Meeting Booking System integrates a full suite of core functionalities for managing institutional meeting room reservations, with each module logically separated and collaboratively coordinated to streamline both user experience and administrative oversight.

#### Entry and Authentication Flow:

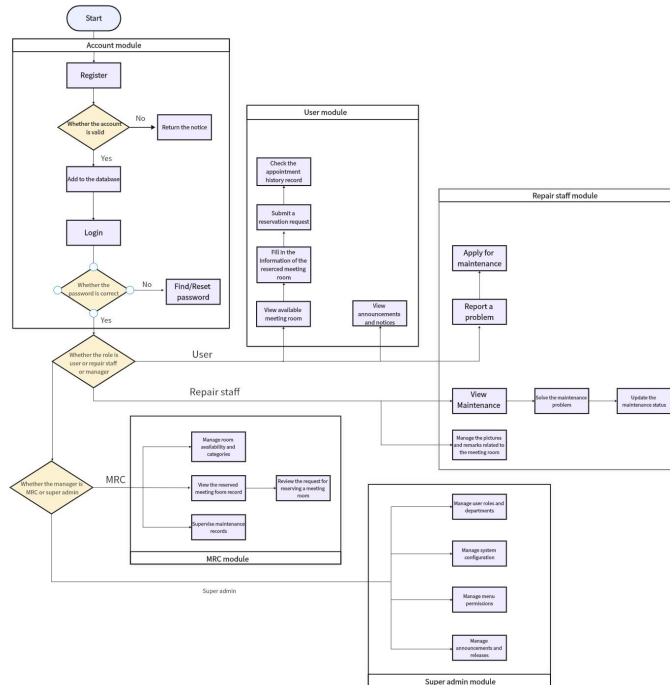


**Figure 5: Process Flow**

Upon accessing the system homepage, users can register by submitting account credentials and basic personal information. Following successful registration and login, the system enforces secure session management, including duplicate-login prevention and session expiration. Based on authenticated roles, users are automatically redirected to their respective dashboards.

### Role-Based Routing and Functional Flow

- **Student / Teacher Users** are directed to a personalized dashboard, where they can browse available meeting rooms via a calendar interface (integrated with FullCalendar), submit and manage booking requests, track booking history, report facility issues, and view system announcements.

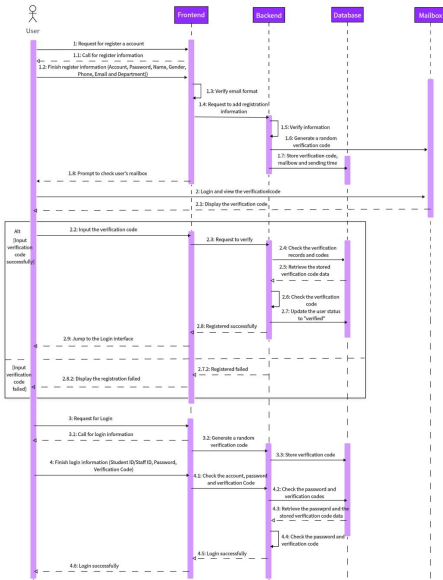


**Figure 6: Role login division flow chart**

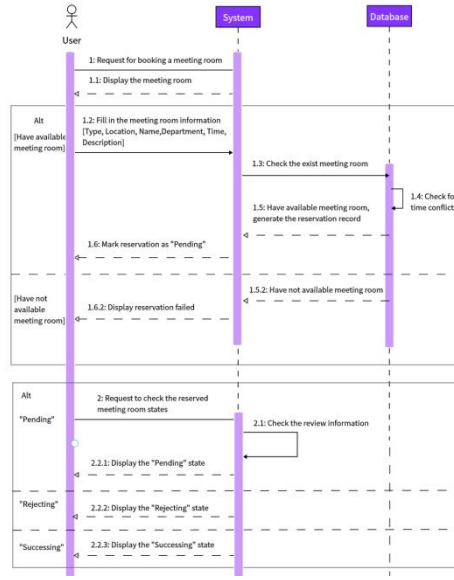
- **MRC (Conference Coordinator)** access an administrative dashboard enabling them to review, approve, or reject booking applications, manage room availability and metadata, monitor room usage trends, oversee repair ticket statuses, and interact with a persistent information pane implemented via Vue 3, Element Plus, and <keep-alive>, ensuring critical approval tasks remain continuously visible.

- **Repair Staffs** are routed to the repair management module, where they can view user-submitted maintenance reports, update repair statuses (e.g., pending, in progress, completed), upload supporting documentation, and prioritize repair workflows based on task urgency.

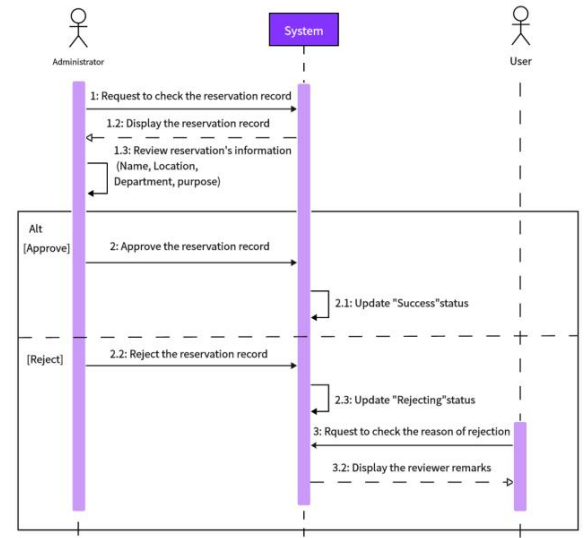
- **Test Admin (Super Admin)** users gain full administrative access, managing system-wide entities including users, departments, roles, and permission menus (sys\_menu), publishing global announcements (sys\_notice), monitoring all booking and repair activities, and operating within a persistent admin workspace enhanced by router caching and <keep-alive> to maintain navigation state and operational continuity.



**Figure 7 : User registration and login**



**Figure 8:User reserved meeting room**



**Figure 9: Administrator reviews meeting room booking records**

This role-driven, modular process flow supports high usability, system reliability, and functional scalability across diverse user scenarios.

### 3.3. Software Support Services

Our Meeting Booking System relies on a suite of software tools and service integrations that collectively enhance development efficiency, system performance, and operational maintainability. These services span across database development, interface navigation, and other supporting services.

#### Database-Related Services

##### • MySQL:

MySQL serves as the primary relational database management system, supporting normalized table structures and high-performance query execution. Foreign key constraints ensure robust referential integrity between core entities such as users, meeting rooms, reservations, and permissions.

##### • MyBatis-Plus + MyBatis-Plus-Join:

We adopted MyBatis-Plus and MyBatis-Plus-Join as our ORM solution to streamline SQL operations. These tools allow concise query writing, auto-generation of boilerplate code, and advanced conditional query support, significantly reducing the complexity and redundancy of database access logic.

##### • Navicat:

Navicat was used during the development phase to visually manage the MySQL database. It enabled collaborative access to table structures, ERD diagrams, test queries, and data population across development environments (Ozar, 2012).

#### Web Navigation Services

The system features a role-adaptive sidebar navigation built with Vue.js 3 and Element Plus, supporting collapsible, hierarchical menus with route-based access control. Navigation state is persistently maintained using Vue Router and <keep-alive> to ensure smooth user experiences across modules. This responsive layout guarantees accessibility across varied screen resolutions and supports interface consistency for students, teachers, MRCs, repair staff, and administrators.

#### Other Supporting Services

##### • Spring Boot Mail Service:

The system integrates Spring Boot's JavaMailSender for email-based verification. During registration, the backend generates a secure, randomized code and sends it to the user's provided email. The mail service is designed to be event-driven and extensible for future use cases such as reservation status notifications and administrative announcements (Deinum, 2024).

##### • Git + Coding.net:

Source code versioning was managed via Git, with remote repositories hosted on Coding.net. Each team member developed within independent branches following a Gitflow-like model, using pull requests for integration and code reviews. Coding.net also supported Scrum task tracking and Wiki documentation (Guthals, 2023).

- **Maven:**

Maven was used as the backend's dependency and build management tool. It helped manage transitive dependencies, resolve version conflicts, and package the Spring Boot application into a JAR file for production deployment. Maven's standard project structure also facilitated better collaboration among backend developers (Varanasi, 2019).

Together, these services form a cohesive foundation that supports rapid development, modular deployment, and stable maintenance throughout the project lifecycle.

### 3.4. Coding Structure and Convention

#### **Project Structure**

The backend project is organized following a conventional multi-layered architecture, with packages segmented into controller, service, mapper, entity, config, and utils, aligned with Model-Service-Controller design principles. Detailed definitions of each directory's responsibilities are documented internally (refer to **Appendix 6, Figure 10 for the Java directory structure**).

The frontend project, built with Vue.js 3 and Vite, adopts a modular structure, separating source code into folders such as views, components, router, assets, store, and api, supporting component-based development and role-based route control.

#### **Method Invocation Convention**

Controllers invoke service methods strictly through defined interfaces, ensuring loose coupling between layers. Each interface may have one or more implementation classes, providing flexibility for logic extension and supporting unit test mockability.

#### **Naming Convention**

We adhered to standardized naming practices for consistency and clarity:

- 1. Methods and variables:** camelCase (e.g., createBookingRequest)
- 2. Class names:** PascalCase (e.g., BookingController)
- 3. Constants:** UPPER\_CASE with underscores (e.g., MAX\_ROOM\_CAPACITY)
- 4. Naming principle:** all names are semantically meaningful and follow the "entity + action" pattern when applicable (e.g., getUserPermissionList)
- 5. Interface implementation classes:** suffixed with Impl (e.g., RoomServiceImpl)

#### **Code Maintenance Standards**

During collaborative development, code maintainability was emphasized:

- 1.** Developers are expected to write self-documenting code with concise and relevant inline comments
- 2.** Prior to any Git commits, unused code segments and outdated comments are cleaned up to preserve repository hygiene
- 3.** Code formatting follows IDE-configured style guides across the team to maintain structural consistency

#### **Task Management and Code Completion**

Unfinished features and pending development tasks are systematically tracked using Coding.net's Scrum board and Wiki documentation, rather than through inline TODO comments. Sprint-based tracking ensures transparent task assignment and facilitates continuous development progress.

By adhering to these standardized structures and conventions, the Meeting Booking System achieves high code readability, efficient collaboration, and scalable maintenance across the development lifecycle.

### 3.5. Software Configuration and Production Environment

#### Software Configuration

The Meeting Booking System adopts a modular, layered configuration to ensure responsive user interaction, secure data handling, and scalable deployment.

- **Frontend Configuration:**

The frontend is developed using Vue.js 3 with TypeScript and Vite, structured as a Single Page Application (SPA). Element Plus ensures consistent UI components, while Vue Router and Axios handle routing and backend communication. ECharts and FullCalendar support visualization and scheduling (Jansen, Vane and Wolff, 2016). This setup enables a responsive and interactive client-side experience.

- **Backend Configuration:**

The backend uses Spring Boot with an MVC structure, supported by MyBatis-Plus for ORM and Druid for connection pooling. Sa-Token and Spring Security Crypto manage authentication, authorization, and secure user session control. The backend ensures secure, scalable, and efficient business processing.

- **Database Configuration:**

MySQL is employed with a 3NF-compliant schema, ensuring data normalization and integrity through foreign key constraints. Navicat assists in schema management and query debugging during development. The database layer guarantees reliable data consistency and scalability.

#### Production Environment

|                |                                                                   |
|----------------|-------------------------------------------------------------------|
| System         | Ubuntu 22.04 64x                                                  |
| Processor      | Intel ® Xeon ® Platinum Scalable Processors, 4-core (vCPU), 8 GiB |
| Public network | bandwidth 5 Mbps                                                  |
| Java OpenJDK   | version 17.0.14                                                   |
| MySQL          | Version 5.7.27                                                    |

Our system is deployed on the **AliyunServer** (ecs.e-c1m2.xlarge) with enhanced network performance and dedicated VPC. This ensures high availability, secure isolation, and stable operation for our application.

## 4. Software testing

We wrote code to test the functionality of the software, mainly focusing on unit and integration tests, as shown in **APPENDIX 9**. Acceptance tests were performed by using the web, and all of the tests results are **'pass'**.

### 4.1. Unit Test

Unit tests verify isolated functionality such as logic correctness and method response, ensuring that each unit of code works accurately. We used the JUnit framework, Mockito ,and other simulation tools to emulate the service layer behavior so that the controller methods are tested independently of the Spring Boot lifecycle to ensure that the individual functions and components work as expected before integration.We focus on **User Registration**.

**Test Description:**Create a simulated user object containing a username and password, then by simulating the behavior of the user's service layer (assuming that the user does not exist and that the save operation is successful), call the registration interface and check that the relevant service methods (query user, save user, and roles) have been called correctly and as expected.

**Test Code:**

```
@Test
void testRegister_Success() {
    // Arrange:Set up test data and mock dependencies
    SysUser newUser = new SysUser();
    newUser.setUserId(1L); // Set user ID
    newUser.setUsername("testUser"); // Set username
    newUser.setPassword("123456"); // Set password

    // Mock service layer responses to simulate scenarios:
    when(sysUserService.getById(any(Long.class))).thenReturn(null); // User does not exist
    when(sysUserService.save(any(SysUser.class))).thenReturn(true); // User saved successfully
    when(sysUserRoleService.save(any(SysUserRole.class))).thenReturn(true); // User-role saved successfully

    // Act:Execute the method under test
    ResultVo result = sysUserController.register(newUser);

    // Assert: Verify the expected outcomes
    assertEquals(200, result.getCode()); // Check status code
    assertEquals("Registration successful!", result.getMsg()); // Verify success message

    // Verify service layer interactions:
    verify(sysUserService, times(1)).getById(any(Long.class)); //User existence check was performed
    verify(sysUserService, times(1)).save(any(SysUser.class)); //User creation was attempted
    verify(sysUserRoleService, times(1)).save(any(SysUserRole.class)); //Role assignment was attempted
}
```

**Figure 11/APPENDIX 11 (clearer): Unit test of user registration**

## Test Results:

| Test Name   | CPT202 Unit Test-System User Controller Test |                                  |                                                                         |                                                                         |        |
|-------------|----------------------------------------------|----------------------------------|-------------------------------------------------------------------------|-------------------------------------------------------------------------|--------|
| Test Date   | 2025.4.28                                    |                                  |                                                                         |                                                                         |        |
| Test Number | Test Case                                    | Data (user_id,username,password) | Expected Result                                                         | Actual Result                                                           | Status |
| 1           | Register_Success                             | 1,testUser,123456                | code=200, message="Registration successful"                             | code=200, message="Registration successful"                             | Pass   |
| 2           | Register_UserAlreadyExists                   | 1,existingUser,null              | code=500, message="The user already exists. Please try a different one" | code=500, message="The user already exists. Please try a different one" | Pass   |
| 3           | Register_SaveUserFails                       | 1,testUser,password123           | code=500, message="Registration failed. Please try again"               | code=500, message="Registration failed. Please try again"               | Pass   |
| 4           | Register_SaveUserRoleFails                   | 1,testUser,123456                | code=500, message="Registration failed. Please try again"               | code=500, message="Registration failed. Please try again"               | Pass   |

Figure 12:Unit test results

## 4.2. Integration Test

Integration testing verifies the correctness of interfaces, interactions and co-operation between multiple modules, which includes controller-to-service communication, database interaction and data validation mechanisms.

We focus on **new meeting room reservation** and **mail system integration**.

**Test Case:**New meeting room reservation

**Test Description:**We used the Spring Boot framework and JUnit testing framework, as well as the MockMvc tool to simulate HTTP requests and implement JSON serialization through ObjectMapper to test an interface for booking meeting rooms. First we constructed the request object containing the title, remark and start/end time, sent a POST request to the interface and converted the request body to JSON format; then we verified that the HTTP response status code was as expected and checked the correctness of specific fields in the returned data.

**Test Code:**

```
@SpringBootTest
@AutoConfigureMockMvc
public class MeetingBookIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;

    @Test
    public void testCreateMeetingBook() throws Exception {
        // Arrange: Prepare test data
        MeetingBookRequest request = new MeetingBookRequest();
        request.setTitle("Team Meeting"); // Set meeting title
        request.setRemark("Discuss project updates"); // Add meeting description
        request.setStartTime(LocalDateTime.parse("2025-04-20T10:00:00")); // Define start time
        request.setEndTime(LocalDateTime.parse("2025-04-20T11:00:00")); // Define end time

        // Act & Assert: Execute request and validate response
        mockMvc.perform(post("/api/meeting-book") // POST request to endpoint
            .contentType(MediaType.APPLICATION_JSON) // Set JSON content type
            .content(objectMapper.writeValueAsString(request))) // Serialize request body
            .andExpect(status().isOk()) // Verify 200 OK status
            .andExpect(jsonPath("$.isNotEmpty()", is(true))) // Check non-empty response
            .andExpect(jsonPath("$.roomId", notNullValue())) // Ensure "id" exists in response
            .andExpect(jsonPath("$.title", is("Team Meeting"))) // Check for title
            .andExpect(jsonPath("$.remark", is("Discuss project updates"))); // Check for remark
    }
}
```

Figure 13/**APPENDIX 12 (clearer)** : Integration test of new meeting room

## Test Results:

| Test Name   | CPT202 Integration Test-Meeting Book Integration Test |                                                                                                                                                                                           |                                                                |                                  |        |
|-------------|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|----------------------------------|--------|
| Test Date   | 2025.4.28                                             |                                                                                                                                                                                           |                                                                |                                  |        |
| Test Number | Test Case                                             | Data                                                                                                                                                                                      | Expected Result                                                | Actual Result                    | Status |
| 1           | CreateMeetingBook                                     | Request body: POST /api/meeting-book<br>Parameters: title="Team Meeting",<br>remark="Discuss project updates",<br>startTime="2025-04-20T10:00",<br>endTime="2025-04-20T11:00"             | code=200,Response Body Non Empty (jsonPath("\$.isNotEmpty()")) | code=200,Response Body Non Empty | Pass   |
| 2           | GetMeetingBookById                                    | Request: GET /api/meeting-book/1<br>Assuming that the meeting record with ID 1 exists                                                                                                     | code=200,Response Body Non Empty                               | code=200,Response Body Non Empty | Pass   |
| 3           | UpdateMeetingBook                                     | Request body: title="Updated Meeting",<br>remark="Discuss project updates",<br>startTime="2025-04-20T10:00",<br>endTime="2025-04-20T11:00:00". Send<br>PUT request to /api/meeting-book/1 | code=200,Response Body Non Empty                               | code=200,Response Body Non Empty | Pass   |
| 4           | DeleteMeetingBook                                     | Send a DELETE request to /api/meeting-book/1, assuming that a meeting record                                                                                                              | code=200,Response Body Non Empty                               | code=200,Response Body Non Empty | Pass   |
| 5           | GetAllMeetingBooks                                    | Send GET request to /api/meeting-book                                                                                                                                                     | code=200,Response Body Non Empty                               | code=200,Response Body Non Empty | Pass   |

Figure 14:Integration test results

## 4.3. Acceptance Test

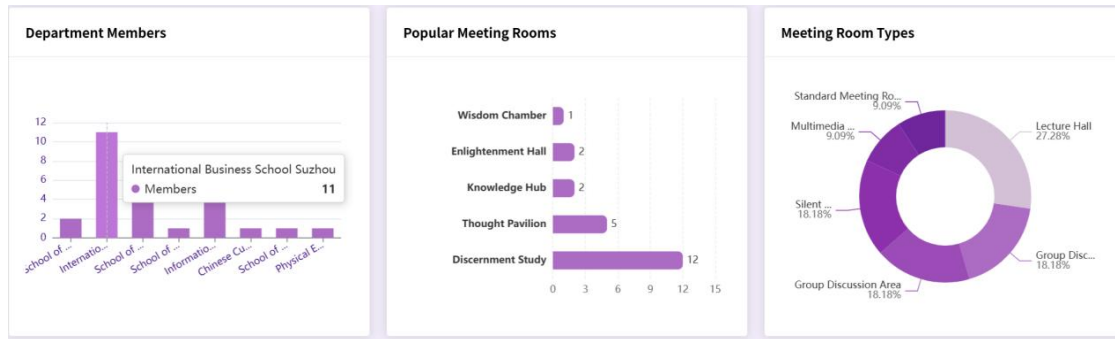
Acceptance testing is to verify that the system under the software meets the business requirements and can function properly in real applications and be accepted by users (Dong, 2013). Our acceptance testing includes business process testing, compatibility and user experience testing, security and performance testing, and accessibility testing.

|             |                                                   |                                                                                  |                               |        |  |
|-------------|---------------------------------------------------|----------------------------------------------------------------------------------|-------------------------------|--------|--|
| Test Name   | CPT202 Acceptance Test                            |                                                                                  |                               |        |  |
| Test Date   | 2025.4.23                                         |                                                                                  |                               |        |  |
| Test Number | Test Input                                        | Expected Result                                                                  | Actual Result                 | Result |  |
| 1           | Select correct range of beginning and ending date | Display correct result of selected transactions with volume and transaction      | Actual result displayed       | Pass   |  |
| 2           | Select incorrect range of date                    | Display error message 'Invalid setting, please check it' in red                  | Error message displayed       | Pass   |  |
| 3           | Press 'generate' button with expected time range  | Generate two figures of statistical information (order volume and total revenue) | Figures generated as expected | Pass   |  |

Figure 15: Test Case ID:PBI-032 Users cannot book for more than 8 hours

In addition, the user interface (UI) should enable users to use the web pages and complete conference room bookings efficiently, easily and pleasantly.Acceptance testing is not just about checking that the UI elements 'look

right’, but more importantly verifying that the UI ‘works right’. This is an example of user and meeting room visual charts.



**Figure 16: User and meeting room usage dashboard**

## 5. Conclusion and future work

So far, our group has made significant progress in developing a functional and efficient Meeting Booking System. We translated project requirements into a well-structured system design, adopting the MVC architecture to achieve a decoupled and maintainable framework. The frontend and backend, built with Vue.js and Spring Boot, deliver a responsive user experience and robust server-side processing. A high-level database design ensures data integrity and consistency, while software modules tailored to different user roles (Students, Teachers, MRC, Repair Staff, and Super Admin) support operations such as room booking, availability viewing, repair requests, and resource management. The integration of software support services has further enhanced system functionality and reliability. Overall, our system addresses key limitations of traditional booking methods.

Despite our achievements, several challenges and issues have arisen during the implementation of the project. First, ensuring seamless integration between different modules is a complex task that requires careful coordination and testing to resolve any compatibility issues. So far, our web pages have only been adapted for computers, and the performance on cell phones needs to be improved. Secondly, optimizing the system performance to handle many concurrent users and bookings is still a task that needs further attention, in our envisioned scenario, a professional operator is required to approve and control the booking of meeting rooms, and we hope that this function can be fully automated in the future. Thirdly, a few minor bugs and usability issues were identified during the testing phase that needed to be addressed to improve the user experience. Finally, as the project progressed, there were some changes to the requirements, which required some adjustments to the initial plan by adapting and incorporating these changes into the design and implementation process.

Looking ahead, our team plans to continue to improve and enhance the online conference booking system. Next, we will focus on optimizing system performance, improving its response speed and stability under high concurrency conditions, including further optimization of the database, code structure and server processing capabilities. We will also conduct more comprehensive testing work, such as unit testing, integration testing and acceptance testing, to promptly discover and fix potential problems and ensure that the system runs more stably and reliably. At the same time, we hope to better understand everyone's feelings and needs during the operation by collecting user feedback. Based on this feedback, we will continue to improve system functions and user interface design, such as finding more suitable optimization directions for users through A/B testing to improve the overall user experience and satisfaction. In addition, we also plan to explore adding some new features, such as docking with video conferencing platforms, providing more detailed resource usage analysis reports, etc., to further enrich the application scenarios of the system and make conference room management more intelligent and efficient. Through continuous improvement and efforts, we hope to make this conference booking system truly meet the actual needs of the school, improve the level of resource management, and provide a reliable reference for the construction of similar systems in the future.

## 6. References

Amuthan Ganeshan (2016) *Spring MVC: Beginner's Guide - Second Edition*. Birmingham: Packt Publishing. Available at: <https://search.ebscohost-com-s.elink.xjtlu.edu.cn:443/login.aspx?direct=true&db=nlebk&AN=1295342&site=eds-live&scope=site> (Accessed: 26 April 2025).

Deinum, M. (2024) *Java Enterprise Services*. Berkeley, CA: Apress. doi:10.1007/979-8-8688-0113-6\_7.

DuBois, P. (2013) *MySQL*. Fifth edition. Addison-Wesley. Available at: <https://search.ebscohost-com-s.elink.xjtlu.edu.cn:443/login.aspx?direct=true&db=cat01010a&AN=xjtlu.0000506143&site=eds-live&scope=site> (Accessed: 26 April 2025).

Farouk, M. (2017) *Infrastructure Software Modules for Enterprises. [electronic resource] : Flexible Software Systems, Module Use-Cases, and Wireframes*. Apress. Available at: <https://search.ebscohost.com/login.aspx?direct=true&db=cat01010a&AN=xjtlu.0001289263&site=eds-live&scope=site> (Accessed: 26 April 2025).

Gökhan Ozar (2012) *MySQL Management and Administration with Navicat : Master the Tools You Thought You Knew and Discover the Features You Never Knew Existed with This Book and Ebook*. [Place of publication not identified]: Packt Publishing. Available at: <https://search.ebscohost.com/login.aspx?direct=true&db=nlebk&AN=486649&site=eds-live&scope=site> (Accessed: 26 April 2025).

Guthals, S. (2023) *GitHub*. 2nd edition. John Wiley & Sons, Inc. (For dummies). Available at: <https://search.ebscohost.com/login.aspx?direct=true&db=cat01010a&AN=xjtlu.0002445372&site=eds-live&scope=site> (Accessed: 26 April 2025).

Heckler, M. (2021) *Spring boot : up and running : building Cloud Native Java and Kotlin applications*. First Edition. O'Reilly Meida, Inc. Available at: <https://search.ebscohost-com-s.elink.xjtlu.edu.cn:443/login.aspx?direct=true&db=cat01010a&AN=xjtlu.0001901260&site=eds-live&scope=site> (Accessed: 26 April 2025).

Jansen, R.H., Vane, V. and Wolff, I.G. de (2016) *TypeScript: Modern JavaScript Development*. Birmingham, UK: Packt Publishing. Available at: <https://search.ebscohost-com-s.elink.xjtlu.edu.cn:443/login.aspx?direct=true&db=nlebk&AN=1444389&site=eds-live&scope=site> (Accessed: 26 April 2025).

Li, Y.Z. *et al.* (2020) 'Research and Application of Template Engine for Web Back-end Based on MyBatis-Plus', *Procedia Computer Science*, 166, pp. 206–212. doi:10.1016/j.procs.2020.02.052.

Mendoza, D. and Piedra, N. (2020) 'TutNorBD: Assistant for teaching and learning process of relational database normalization up to 3NF from a universal table', *2020 XV Conferencia Latinoamericana de Tecnologías de Aprendizaje (LACLO), Tecnologías de Aprendizaje (LACLO), 2020 XV Conferencia Latinoamericana de*, pp. 1–8. doi:10.1109/LACLO50806.2020.9381184.

Sanchez, L.M. *et al.* (2019) 'Smart Meeting Room Management System Based on Real-Time Occupancy', *2019 IV Jornadas Costarricenses de Investigación en Computación e Informática (JoCICI), Investigación en Computación e Informática (JoCICI), 2019 IV Jornadas Costarricenses de*, pp. 1–6. doi:10.1109/JoCICI48395.2019.9105174.

Shavin, M. (2023) *Learning Vue : core concepts and practical patterns for reusable, composable, and scalable user interfaces*. First edition. O'Reilly Media, Inc. Available at: <https://search.ebscohost-com->

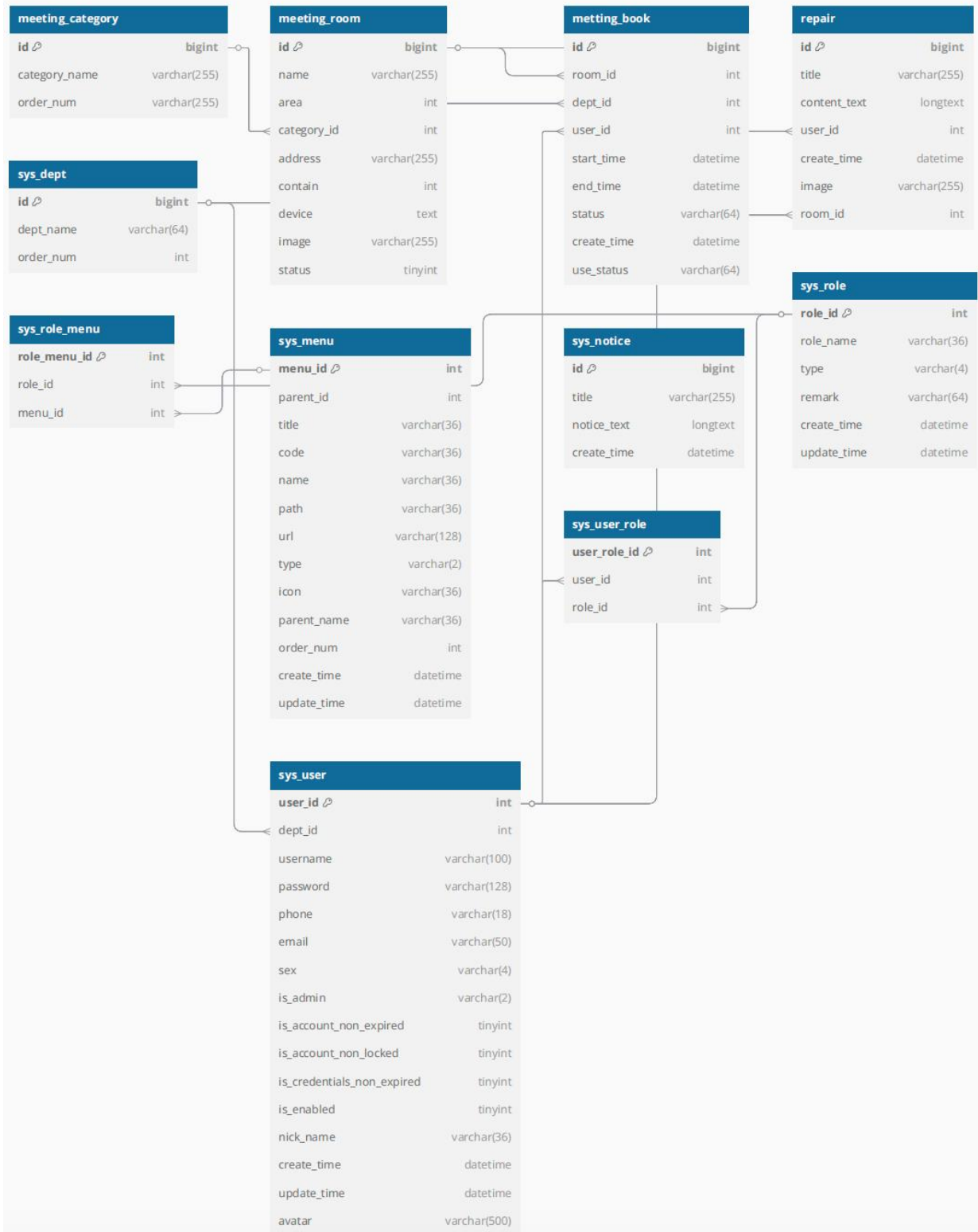
s.elink.xjtlul.edu.cn:443/login.aspx?direct=true&db=cat01010a&AN=xjtlul.0002533318&site=eds-live&scope=site (Accessed: 26 April 2025).

Shili, S., et al., 2009. *Design of the Booking Meeting System on the Electronic Commercial Manageability*. *Journal of Liaoning University (Natural Science Edition)*, 36(1), pp.59–63. doi:10.3969/j.issn.1000-5846.2009.01.017.

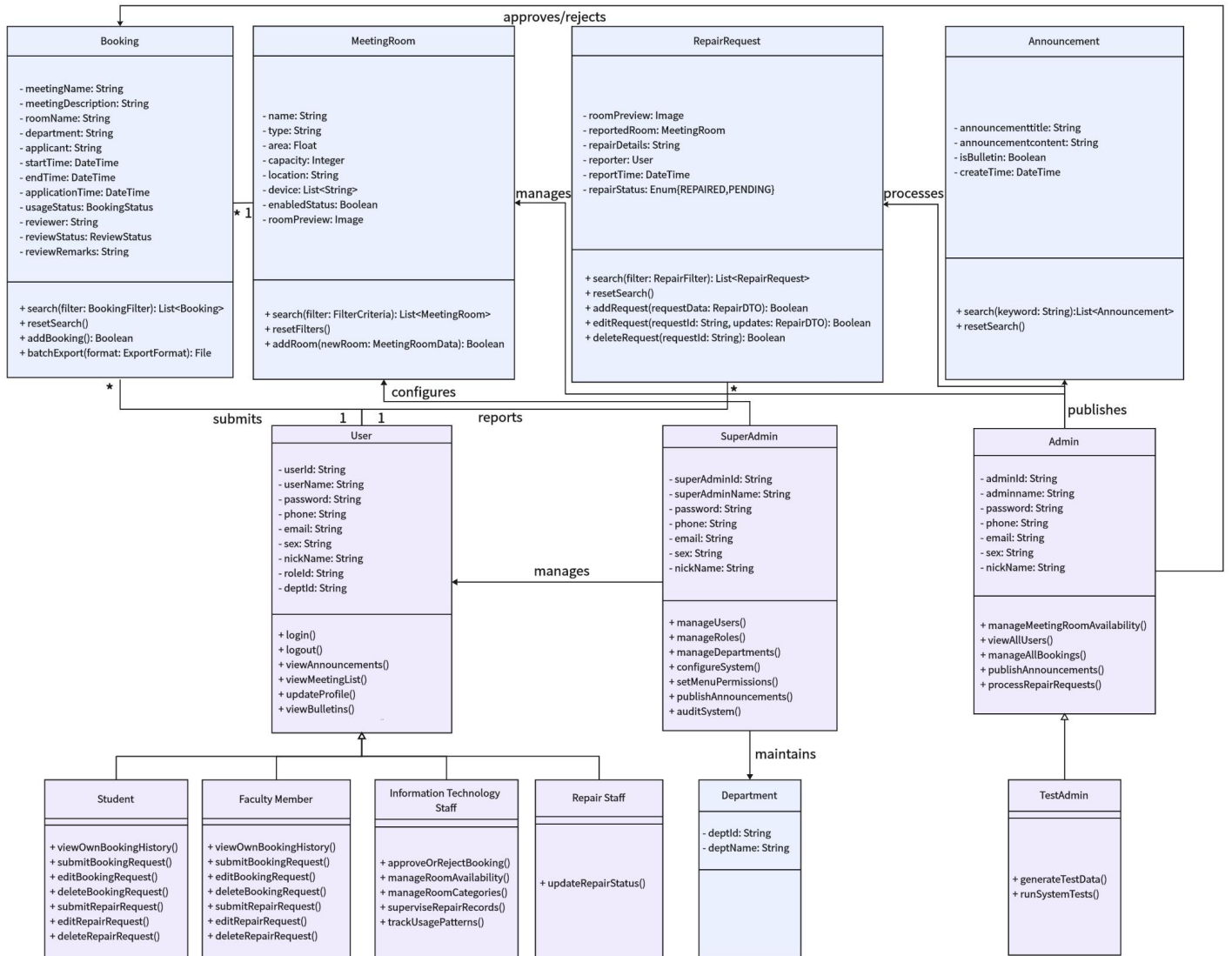
Varanasi, B. (2019) *Introducing Maven. [electronic resource] : A Build Tool for Today's Java Developers*. 2nd ed. 2019. Apress. Available at:  
<https://search.ebscohost.com/login.aspx?direct=true&db=cat01010a&AN=xjtlul.0001649509&site=eds-live&scope=site> (Accessed: 26 April 2025).

## 7. Appendix

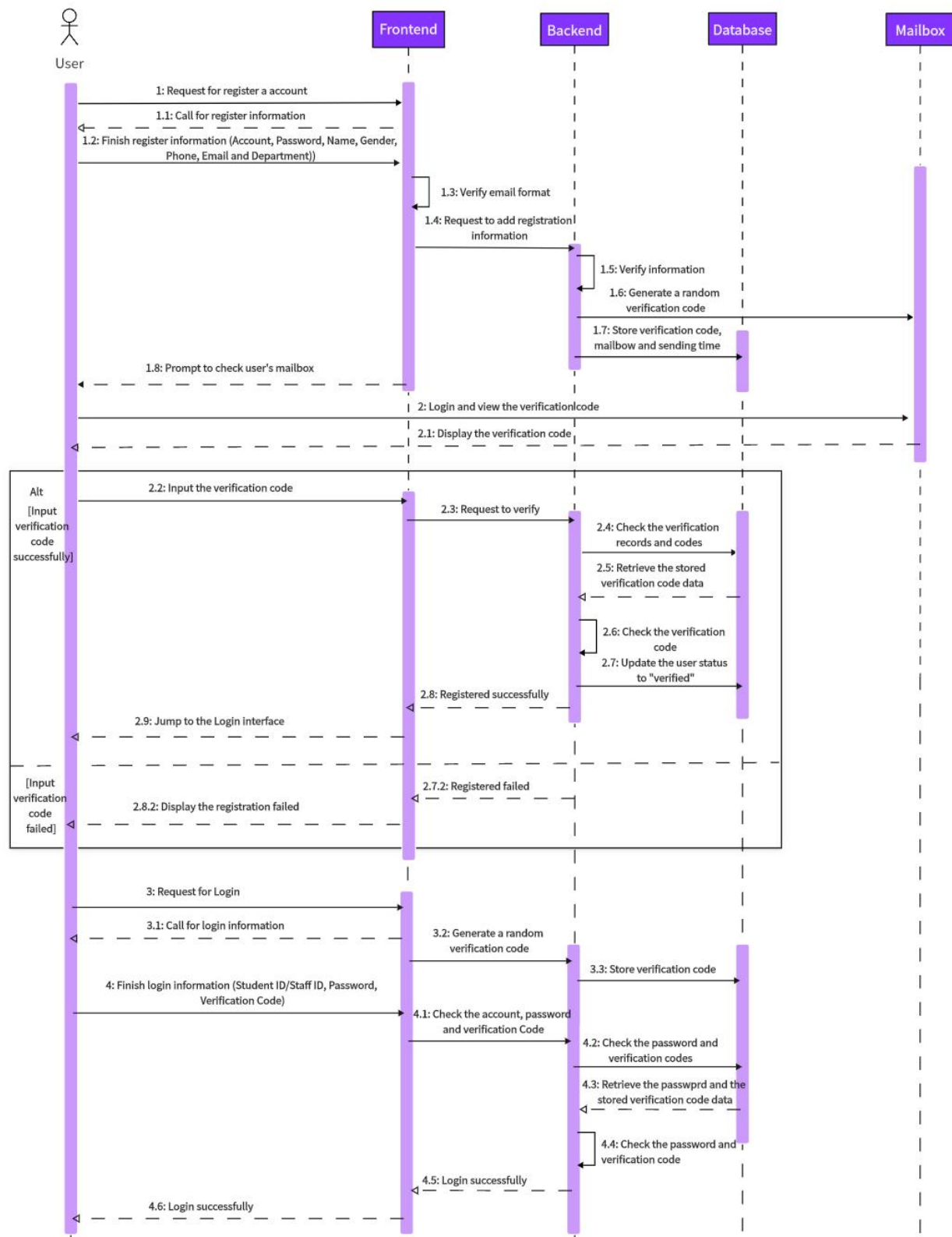
**Appendix 1- Figure 3: ER diagram**



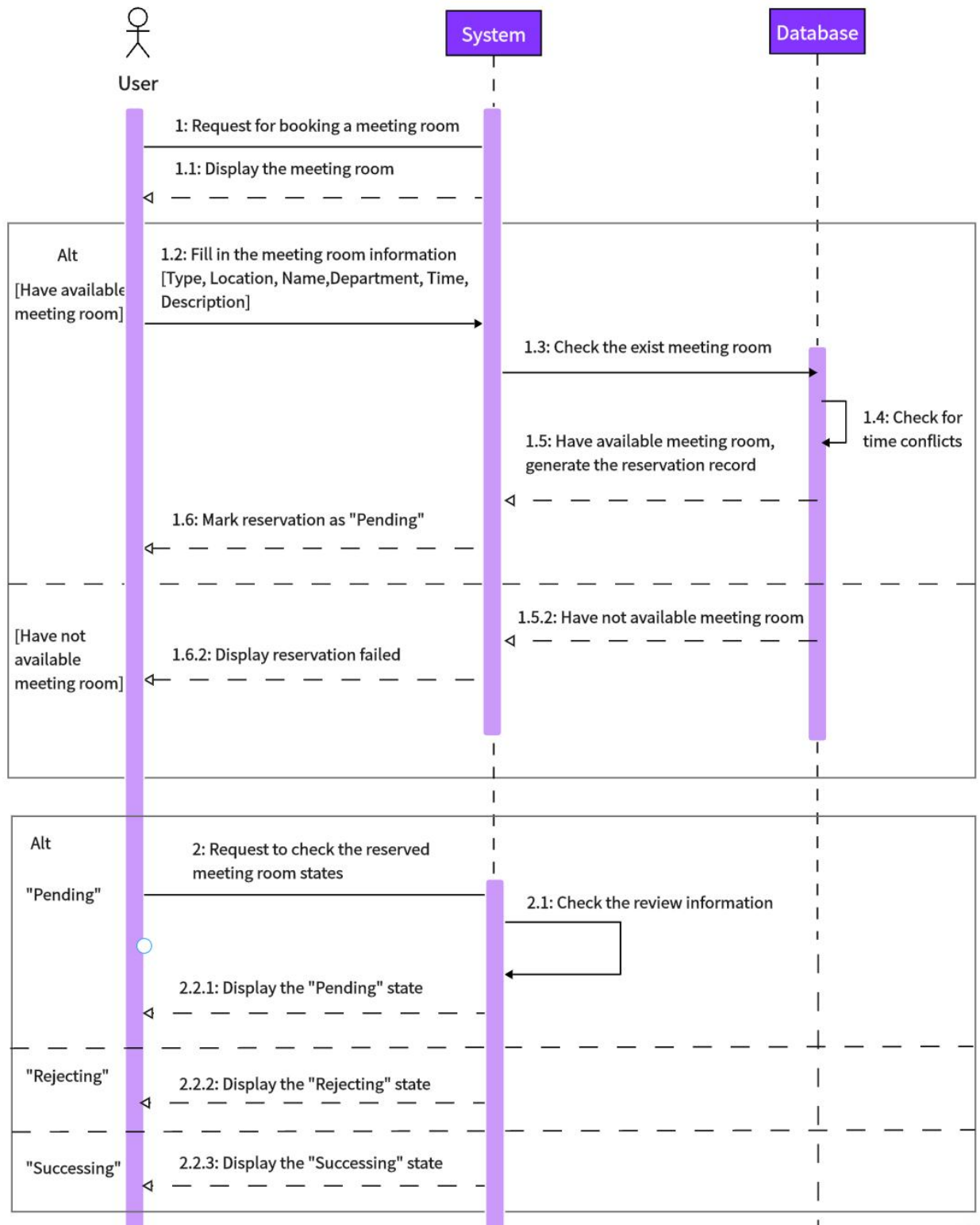
**Appendix 2- Figure 4: Software Modules Design**



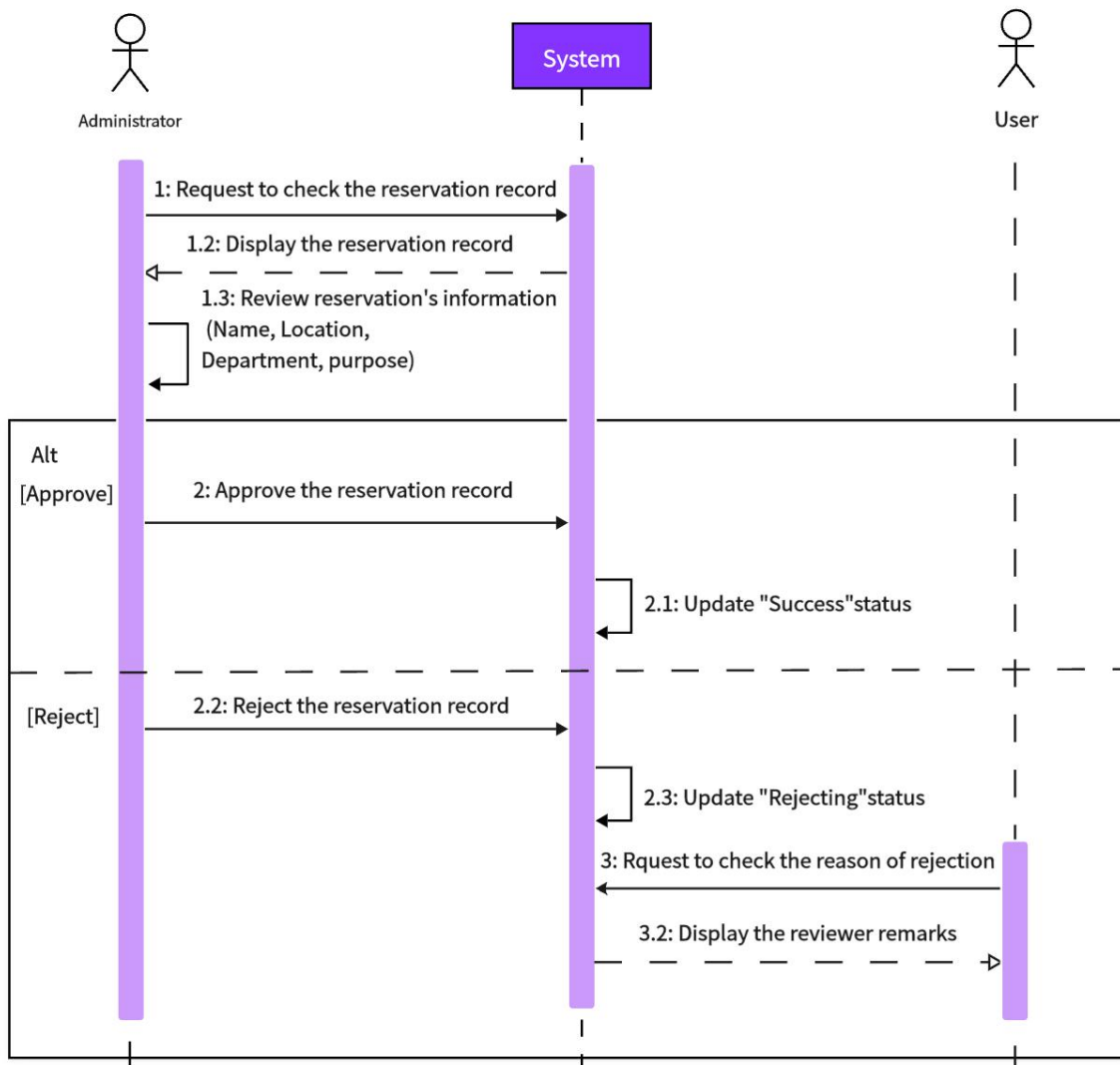
**Appendix 3- Figure 6: Time series diagram: User registration and login**



**Appendix 4- Figure 7: Time series diagram: User reserved meeting room**



**Appendix 5- Figure 8: Time series diagram: Administrator reviews meeting room booking records**



**Appendix 6- Figure 9: JAVA directory**

```

---/src/main/java/com/group17/CPT202_Group17_Backend
|---/config: Configuration class storage location
|---/controller: Control layer
|---/service: Business code layer
|---/entity: Storing database entity classes
|---/mapper: Interaction layer with database

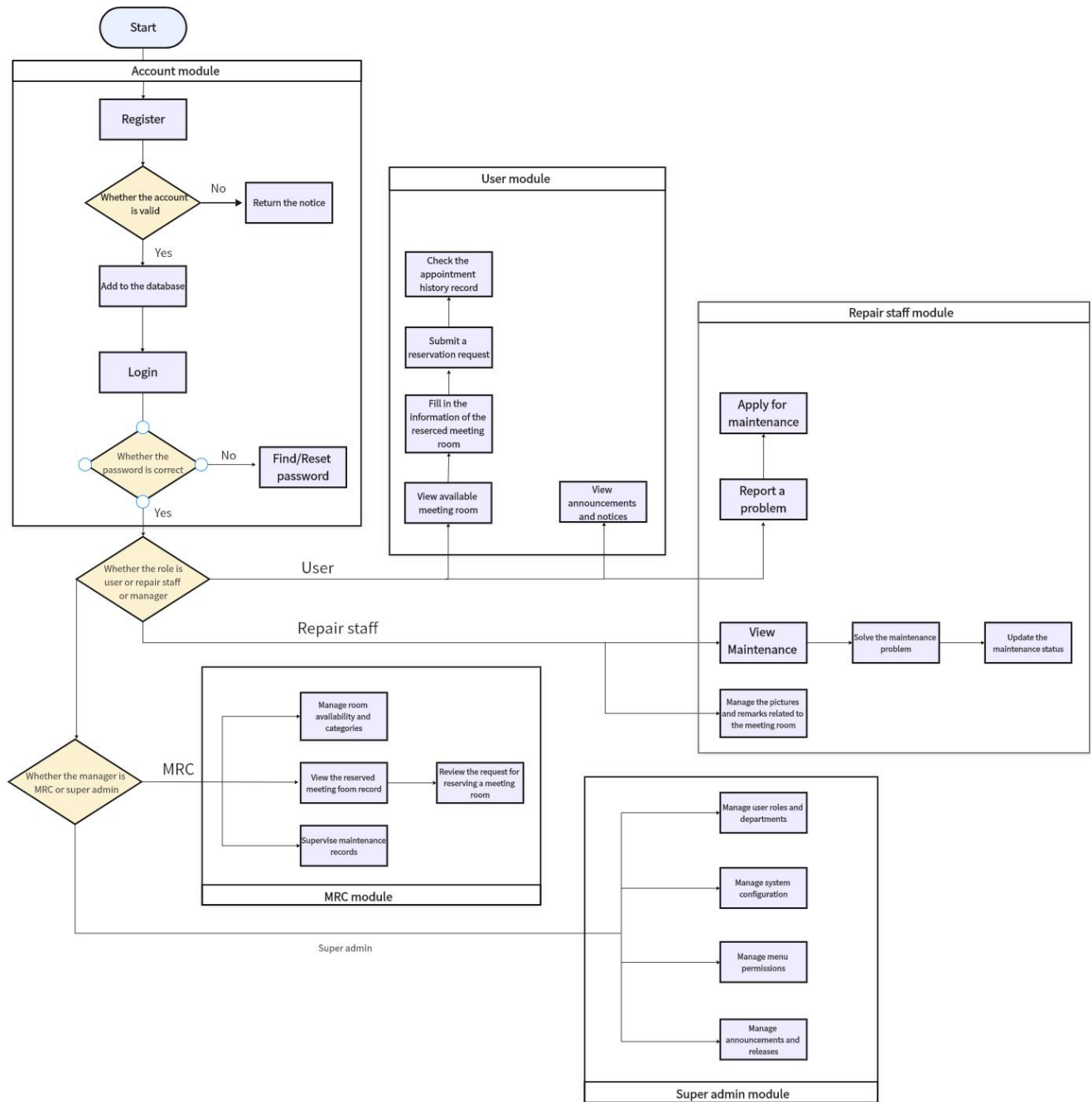
---/src/main/resources: Stores resource files, such as static resource files, configuration
files, page files, etc.
|---/mapper: XML files that store the MyBatis operation database
|---/static: Stores static resources such as CSS styles, images, etc.
|---/templates: Stores template files such as Thymeleaf template files

---/src/test/java/com/group17: Unit test class and integration test class Java files
    
```

## Appendix 7: Sprint Process Report

| Sprint Progress Report |                                  |                                              |               |            |                         |                        |
|------------------------|----------------------------------|----------------------------------------------|---------------|------------|-------------------------|------------------------|
| Project Name           | An online meeting booking system |                                              |               |            |                         |                        |
| Team Number            | 17                               |                                              |               |            |                         |                        |
| Team Name              | Group17                          |                                              |               |            |                         |                        |
| Team Leader            | Xiangyi.Zhu                      |                                              |               |            |                         |                        |
| TA                     | Tong.Zou                         |                                              |               |            |                         |                        |
| Week                   | Sprint                           | PBI                                          | Owener        | Start Date | Expected CompletionDate | Actual Completion Date |
| 1                      | 1                                | Write requirement documentation.             | Xiangyi Zhu   | 2/17/2025  | 2/18/2025               | 2/18/2025              |
|                        | 1                                | Choose technology stack                      | Xiangyi Zhu   | 2/17/2025  | 2/18/2025               | 2/18/2025              |
|                        | 1                                | Initialize project repository                | Xinrong Shi   | 2/17/2025  | 2/18/2025               | 2/18/2025              |
|                        | 1                                | Design user registration page                | Hanqiu Zhang  | 2/19/2025  | 2/21/2025               | 2/21/2025              |
|                        | 1                                | Develop registration API                     | Hanqiu Zhang  | 2/19/2025  | 2/21/2025               | 2/21/2025              |
|                        | 1                                | Implement email verification.                | Xiangyi Zhu   | 2/19/2025  | 2/21/2025               | 2/21/2025              |
|                        | 1                                | Design login page                            | Xinyuan Zhang | 2/19/2025  | 2/21/2025               | 2/21/2025              |
|                        | 1                                | Develop login API                            | Xiangyi Zhu   | 2/22/2025  | 2/23/2025               | 2/23/2025              |
|                        | 1                                | Create user profile editing page             | Nan Lin       | 2/22/2025  | 2/23/2025               | 2/23/2025              |
|                        | 1                                | Configure admin menu permissions             | Nan Lin       | 2/22/2025  | 2/23/2025               | 2/23/2025              |
| 2                      | 2                                | Load dynamic menus based on roles            | Zimo Zheng    | 2/24/2025  | 2/27/2025               | 2/27/2025              |
|                        | 2                                | Develop permission middleware                | Xinrong Shi   | 2/24/2025  | 2/27/2025               | 2/27/2025              |
|                        | 2                                | Design room category management              | Nan Lin       | 2/24/2025  | 2/27/2025               | 2/27/2025              |
|                        | 2                                | Develop room CRUD APIs                       | Zimo Zheng    | 2/24/2025  | 2/27/2025               | 2/27/2025              |
|                        | 2                                | Add room status labels (available/disabled)  | Zwei Zhou     | 2/24/2025  | 2/27/2025               | 2/27/2025              |
|                        | 2                                | Design room search/filter UI                 | Zwei Zhou     | 2/24/2025  | 2/27/2025               | 2/27/2025              |
|                        | 2                                | Implement time conflict detection (back-end) | Xiangyi Zhu   | 28/2/2025  | 3/2/2025                | 3/2/2025               |
|                        | 2                                | Develop booking approval workflow            | Jiayi Wang    | 28/2/2025  | 3/2/2025                | 3/2/2025               |
|                        | 2                                | Create user booking history page             | Jiayi Wang    | 28/2/2025  | 3/2/2025                | 3/2/2025               |
|                        | 2                                | Encrypt passwords                            | Xinrong Shi   | 28/2/2025  | 3/2/2025                | 3/2/2025               |
| 3                      | 3                                | Prevent SQL injection                        | Nan Lin       | 3/3/2025   | 3/9/2025                | 3/9/2025               |
|                        | 3                                | Cache sessions with Redis                    | Xiangyi Zhu   | 3/3/2025   | 3/9/2025                | 3/9/2025               |
|                        | 3                                | Optimize database indexes                    | Hanqiu Zhang  | 3/3/2025   | 3/9/2025                | 3/9/2025               |
|                        | 3                                | Add email notifications                      | Xiangyi Zhu   | 3/3/2025   | 3/9/2025                | 3/9/2025               |
|                        | 3                                | Log admin operations                         | Jiayi Wang    | 3/3/2025   | 3/9/2025                | 3/9/2025               |
|                        | 3                                | Build data dashboards                        | Xinrong Shi   | 3/3/2025   | 3/9/2025                | 3/9/2025               |
|                        | 3                                | Write unit tests (JUnit)                     | Xinrong Shi   | 3/3/2025   | 3/9/2025                | 3/9/2025               |
|                        | 3                                | Test APIs with Postman                       | Zwei Zhou     | 3/3/2025   | 3/9/2025                | 3/9/2025               |
|                        | 3                                | Dockerize backend/frontend                   | Jiayi Wang    | 3/3/2025   | 3/9/2025                | 3/9/2025               |
|                        | 3                                | Configure Nginx and HTTPS                    | Hanqiu Zhang  | 3/3/2025   | 3/9/2025                | 3/9/2025               |
| 4                      | 3                                | Set up CI/CD pipeline                        | Hanqiu Zhang  | 3/3/2025   | 3/9/2025                | 3/9/2025               |
|                        | 3                                | Monitor server logs (ELK Stack)              | Nan Lin       | 3/3/2025   | 3/9/2025                | 3/9/2025               |
|                        | 4                                | Collect user feedback                        | Zimo Zheng    | 3/10/2025  | 3/12/2025               | 3/12/2025              |
|                        | 4                                | Plan version updates (V1.0 → V1.2)           | Xiangyi Zhu   | 3/10/2025  | 3/12/2025               | 3/12/2025              |
|                        | 4                                | Design password reset page                   | Zimo Zheng    | 3/10/2025  | 3/12/2025               | 3/12/2025              |
|                        | 4                                | Develop password reset API                   | Xiangyi Zhu   | 3/10/2025  | 3/12/2025               | 3/12/2025              |
|                        | 4                                | Create admin dashboard layout                | Zimo Zheng    | 3/10/2025  | 3/12/2025               | 3/12/2025              |
|                        | 4                                | Add room image upload functionality          | Xinyuan Zhang | 3/13/2025  | 3/16/2025               | 3/16/2025              |
|                        | 4                                | Implement booking cancellation logic         | Xinyuan Zhang | 3/13/2025  | 3/16/2025               | 3/16/2025              |
|                        | 4                                | Validate user input (front-end/back-end)     | Zimo Zheng    | 3/13/2025  | 3/16/2025               | 3/16/2025              |
| 5                      | 4                                | Handle token expiration (JWT)                | Zimo Zheng    | 3/13/2025  | 3/16/2025               | 3/16/2025              |
|                        | 4                                | Generate monthly usage reports               | Xinyuan Zhang | 3/13/2025  | 3/16/2025               | 3/16/2025              |
|                        | 5                                | Integrate third-party calendars              | Zwei Zhou     | 3/17/2025  | 3/20/2025               | 3/20/2025              |
|                        | 5                                | Monitor system performance                   | Xiaoya Li     | 3/17/2025  | 3/20/2025               | 3/20/2025              |
|                        | 5                                | Backup database regularly                    | Xinyuan Zhang | 3/17/2025  | 3/20/2025               | 3/20/2025              |
|                        | 5                                | Implement rate limiting                      | Jiayi Wang    | 3/17/2025  | 3/20/2025               | 3/20/2025              |
|                        | 5                                | Design error handling pages                  | Xinyuan Zhang | 3/17/2025  | 3/20/2025               | 3/20/2025              |
|                        | 5                                | Test cross-browser compatibility             | Xinrong Shi   | 3/17/2025  | 3/20/2025               | 3/20/2025              |
|                        | 5                                | Optimize mobile responsiveness               | Zwei Zhou     | 3/21/2025  | 3/23/2025               | 3/23/2025              |
|                        | 5                                | Conduct load testing                         | Xiaoya Li     | 3/21/2025  | 3/23/2025               | 3/23/2025              |
| 6                      | 6                                | Document API endpoints                       | Zwei Zhou     | 3/24/2025  | 3/30/2025               | 3/30/2025              |
|                        | 6                                | Localize UI for multilingual support         | Jiayi Wang    | 3/24/2025  | 3/30/2025               | 3/30/2025              |
|                        | 6                                | Add keyboard navigation support              | Hanqiu Zhang  | 3/24/2025  | 3/30/2025               | 3/30/2025              |
|                        | 6                                | Implement dark mode toggle                   | Xiaoya Li     | 3/24/2025  | 3/30/2025               | 3/30/2025              |
|                        | 6                                | Audit code for security vulnerabilities      | Xinrong Shi   | 3/24/2025  | 3/30/2025               | 3/30/2025              |
|                        | 6                                | Update dependencies regularly                | Xiaoya Li     | 3/24/2025  | 3/30/2025               | 3/30/2025              |
| 7                      | 6                                | Train admin users                            | Xiaoya Li     | 3/24/2025  | 3/30/2025               | 3/30/2025              |
|                        | 7                                | Create user guides/FAQs                      | Nan Lin       | 3/31/2025  | 4/6/2025                | 4/6/2025               |
|                        | 7                                | Schedule periodic system reviews             | Nan Lin       | 3/31/2025  | 4/6/2025                | 4/6/2025               |

## Appendix 8: Role login division flow chart



Appendix 9 (1): Database Table Structure Documentation

Database Table Structure Documentation

Database Name: meeting-room-system

Version: 1.0.0

Description: Database Design Documentation

Table:meeting\_category (Meeting Room Type)

| No. | Field Name    | Data Type | Length | Decimal | Nullable | Primary Key | Default | Description        |
|-----|---------------|-----------|--------|---------|----------|-------------|---------|--------------------|
| 1   | id            | bigint    | 20     | 0       | N        | Y           |         | Serial number      |
| 2   | category_name | varchar   | 255    | 0       | Y        | N           |         | Category name      |
| 3   | order_num     | varchar   | 255    | 0       | Y        | N           |         | Category order No. |

Table:meeting\_room (Meeting Room Table)

| No. | Field Name  | Data Type | Length | Decimal | Nullable | Primary Key | Default | Description           |
|-----|-------------|-----------|--------|---------|----------|-------------|---------|-----------------------|
| 1   | id          | bigint    | 20     | 0       | N        | Y           |         | Serial number         |
| 2   | name        | varchar   | 255    | 0       | Y        | N           |         | Meeting room name     |
| 3   | area        | int       | 10     | 0       | Y        | N           |         | Area                  |
| 4   | category_id | int       | 10     | 0       | Y        | N           |         | Meeting room category |
| 5   | address     | varchar   | 255    | 0       | Y        | N           |         | Address               |
| 6   | contain     | int       | 10     | 0       | Y        | N           |         | Capacity              |
| 7   | device      | text      | 65535  | 0       | Y        | N           |         | Device                |
| 8   | image       | varchar   | 255    | 0       | Y        | N           |         | Meeting room images   |
| 9   | status      | tinyint   | 0      | 0       | Y        | N           |         | Enabled status        |

Table:meeting\_book (Appointment Management)

| No. | Field Name | Data Type | Length | Decimal | Nullable | Primary Key | Default | Description       |
|-----|------------|-----------|--------|---------|----------|-------------|---------|-------------------|
| 1   | id         | bigint    | 20     | 0       | N        | Y           |         | Serial number     |
| 2   | room_id    | int       | 10     | 0       | Y        | N           |         | Meeting room name |
| 3   | dept_id    | int       | 10     | 0       | Y        | N           |         | User department   |
| 4   | user_id    | int       | 10     | 0       | Y        | N           |         | User id           |

## Appendix 9 (2): Database Table Structure Documentation

|   |             |          |    |   |   |   |  |              |
|---|-------------|----------|----|---|---|---|--|--------------|
| 5 | start_time  | datetime | 19 | 0 | Y | N |  | Start time   |
| 6 | end_time    | datetime | 19 | 0 | Y | N |  | End time     |
| 7 | status      | varchar  | 64 | 0 | Y | N |  | Audit status |
| 8 | create_time | datetime | 19 | 0 | Y | N |  | Create time  |
| 9 | use_status  | varchar  | 64 | 0 | Y | N |  | Use status   |

Table : repair (Repair Management)

| No. | Field Name   | Data Type | Length     | Decimal | Nullable | Primary Key | Default | Description                         |
|-----|--------------|-----------|------------|---------|----------|-------------|---------|-------------------------------------|
| 1   | id           | bigint    | 20         | 0       | N        | Y           |         | Serial number                       |
| 2   | title        | varchar   | 255        | 0       | Y        | N           |         | Title                               |
| 3   | content_text | longtext  | 2147483647 | 0       | Y        | N           |         | Repair details                      |
| 4   | user_id      | int       | 10         | 0       | Y        | N           |         | Person who reports a repair         |
| 5   | create_time  | datetime  | 19         | 0       | Y        | N           |         | Time to report for repair           |
| 6   | image        | varchar   | 255        | 0       | Y        | N           |         | Images of reported repairs          |
| 7   | room_id      | int       | 10         | 0       | Y        | N           |         | Meeting room reported for repairing |

Table: sys\_dept (Department Table)

| No. | Field Name | Data Type | Length | Decimal | Nullable | Primary Key | Default | Description       |
|-----|------------|-----------|--------|---------|----------|-------------|---------|-------------------|
| 1   | id         | bigint    | 20     | 0       | N        | Y           |         | Serial number     |
| 2   | dept_name  | varchar   | 64     | 0       | Y        | N           |         | Department name   |
| 3   | order_num  | int       | 10     | 0       | Y        | N           |         | Department number |

Table : sys\_menu

| No. | Field Name | Data Type | Length | Decimal | Nullable | Primary Key | Default | Description      |
|-----|------------|-----------|--------|---------|----------|-------------|---------|------------------|
| 1   | menu_id    | int       | 10     | 0       | N        | Y           |         | Menu id          |
| 2   | parent_id  | int       | 10     | 0       | Y        | N           |         | Parent id        |
| 3   | title      | varchar   | 36     | 0       | Y        | N           |         | Menu title       |
| 4   | code       | varchar   | 36     | 0       | Y        | N           |         | Permission field |
| 5   | name       | varchar   | 36     | 0       | Y        | N           |         | Route name       |
| 6   | path       | varchar   | 36     | 0       | Y        | N           |         | Routing address  |

### Appendix 9 (3): Database Table Structure Documentation

|    |             |          |     |   |   |   |  |                                             |
|----|-------------|----------|-----|---|---|---|--|---------------------------------------------|
| 7  | url         | varchar  | 128 | 0 | Y | N |  | Component path                              |
| 8  | type        | varchar  | 2   | 0 | Y | N |  | Menu type<br>0:Catalogue 1:Menu<br>2:Button |
| 9  | icon        | varchar  | 36  | 0 | Y | N |  | Menu icon                                   |
| 10 | parent_name | varchar  | 36  | 0 | Y | N |  | Parent menu name                            |
| 11 | order_num   | int      | 10  | 0 | Y | N |  | Serial number                               |
| 12 | create_time | datetime | 19  | 0 | Y | N |  | Create time                                 |
| 13 | update_time | datetime | 19  | 0 | Y | N |  | Update time                                 |

Table: sys\_notice (Announcement Table)

| No. | Field Name  | Data Type | Length     | Decimal | Nullable | Primary Key | Default | Description        |
|-----|-------------|-----------|------------|---------|----------|-------------|---------|--------------------|
| 1   | id          | bigint    | 20         | 0       | N        | Y           |         | Serial number      |
| 2   | title       | varchar   | 255        | 0       | Y        | N           |         | Announcement title |
| 3   | notice_text | longtext  | 2147483647 | 0       | Y        | N           |         | Announcement       |
| 4   | create_time | datetime  | 19         | 0       | Y        | N           |         | Create time        |

Table: sys\_role

| No. | Field Name  | Data Type | Length | Decimal | Nullable | Primary Key | Default | Description |
|-----|-------------|-----------|--------|---------|----------|-------------|---------|-------------|
| 1   | role_id     | int       | 10     | 0       | N        | Y           |         | Role id     |
| 2   | role_name   | varchar   | 36     | 0       | Y        | N           |         | Role name   |
| 3   | type        | varchar   | 4      | 0       | Y        | N           |         | Type        |
| 4   | remark      | varchar   | 64     | 0       | Y        | N           |         | Description |
| 5   | create_time | datetime  | 19     | 0       | Y        | N           |         | Create time |
| 6   | update_time | datetime  | 19     | 0       | Y        | N           |         | Update time |

Table : sys\_role\_menu

| No. | Field Name   | Data Type | Length | Decimal | Nullable | Primary Key | Default | Description |
|-----|--------------|-----------|--------|---------|----------|-------------|---------|-------------|
| 1   | role_menu_id | int       | 10     | 0       | N        | Y           |         | Primary key |
| 2   | role_id      | int       | 10     | 0       | Y        | N           |         | Role id     |
| 3   | menu_id      | int       | 10     | 0       | Y        | N           |         | Menu id     |

## Appendix 9 (4): Database Table Structure Documentation

Table : sys\_user

| No. | Field Name                 | Data Type | Length | Decimal | Nullable | Primary Key | Default | Description                                                               |
|-----|----------------------------|-----------|--------|---------|----------|-------------|---------|---------------------------------------------------------------------------|
| 1   | user_id                    | int       | 10     | 0       | N        | Y           |         | User id                                                                   |
| 2   | dept_id                    | int       | 10     | 0       | Y        | N           |         | Department id                                                             |
| 3   | username                   | varchar   | 100    | 0       | Y        | N           |         | Login account                                                             |
| 4   | password                   | varchar   | 128    | 0       | Y        | N           |         | Login password                                                            |
| 5   | phone                      | varchar   | 18     | 0       | Y        | N           |         | Telephone number                                                          |
| 6   | email                      | varchar   | 50     | 0       | Y        | N           |         | Email address                                                             |
| 7   | sex                        | varchar   | 4      | 0       | Y        | N           |         | Gender                                                                    |
| 8   | is_admin                   | varchar   | 2      | 0       | Y        | N           |         | Whether super administrator<br>1: Yes 0: No                               |
| 9   | is_account_non_expired     | tinyint   | 4      | 0       | Y        | N           |         | Whether the account is expired<br>1 : Not expired<br>0 : Expired          |
| 10  | is_account_non_locked      | tinyint   | 4      | 0       | Y        | N           |         | Whether the account is locked 1 :<br>unlocked<br>0 : locked               |
| 11  | is_credentials_non_expired | tinyint   | 4      | 0       | Y        | N           |         | Whether the password has expired<br>1 : not expired<br>0 : expired        |
| 12  | is_enabled                 | tinyint   | 4      | 0       | Y        | N           |         | Whether the account is available<br>1 : available<br>0 : removes the user |
| 13  | nick_name                  | varchar   | 36     | 0       | Y        | N           |         | Nick name                                                                 |
| 14  | create_time                | datetime  | 19     | 0       | Y        | N           |         | Create time                                                               |

### Appendix 9 (5): Database Table Structure Documentation

|    |             |          |     |   |   |   |  |             |
|----|-------------|----------|-----|---|---|---|--|-------------|
| 15 | update_time | datetime | 19  | 0 | Y | N |  | Update time |
| 16 | avatar      | varchar  | 500 | 0 | Y | N |  | Avatar path |

Table : sys\_user\_role

| No. | Field Name   | Data Type | Length | Decimal | Nullable | Primary Key | Default | Description |
|-----|--------------|-----------|--------|---------|----------|-------------|---------|-------------|
| 1   | user_role_id | int       | 10     | 0       | N        | Y           |         | Primary key |
| 2   | user_id      | int       | 10     | 0       | Y        | N           |         | User id     |
| 3   | role_id      | int       | 10     | 0       | Y        | N           |         | Role id     |

## Appendix 10: Overview of our test part in IDE

3/3 19.7s

web 80.1s

- com.group17.web.auth 13.0s
  - AuthControllerIntegrationTest 5.6s
    - testSendCode\_shouldReturnSuccess() 4.0s
    - testVerifyCode\_shouldReturnSuccessIfCorrect() 1.6s
  - AuthControllerTest 7.4s
    - testSendCode\_ShouldCallSendEmailInternally() 2.0s
    - testSendCode\_EmailContentCorrect() 3.1s
    - testSendCode\_EmailSendFails() 2.2s
- com.group17.web.home.controller 32.3s
  - HomeControllerIntegrationTest 2.6s
    - testGetTotalIntegration() 2.6s
    - testGetEchartsIntegration() 18ms
    - testGetNoticesIntegration() 14ms
  - HomeControllerTest 29.7s
    - testGetTotal() 3.1s
    - testGetEcharts() 6.0s
    - testGetMeetings() 5.5s
    - testGetNotices() 4.8s
    - testGetLastMeeting() 5.1s
    - testGetCategoryBook() 5.2s
- com.group17.web.meeting\_book.controller 4.5s
  - MeetingBookControllerTest 1.7s
    - testApply\_Success() 19ms
    - testApply\_Failure() 5.0ms
    - testApply\_NullMeetingBook() 1.7s
  - MeetingBookIntegrationTest 2.8s
    - testCreateMeetingBook() 16ms
    - testGetMeetingBookById() 2.6s
    - testUpdateMeetingBook() 169ms
    - testDeleteMeetingBook() 14ms
    - testGetAllMeetingBooks() 16ms

- com.group17.web.meeting\_room.controller 431ms
  - MeetingRoomControllerIntegrationTest 46ms
    - testGetRoomListIntegration() 21ms
    - testSearchMeetingRoomIntegration() 13ms
    - testSelectListIntegration() 12ms
  - MeetingRoomControllerTest 385ms
    - testGetRoomList\_NonEmptyList() 279ms
    - testSelectList\_ReturnsDropDownOptions() 58ms
    - testDeleteRoom\_Successful() 48ms
- com.group17.web.sys\_role.controller 342ms
  - SysRoleControllerIntegrationTest 47ms
    - testAddRole\_IntegrationSuccess() 29ms
    - testGetRoleList\_Success() 8.0ms
    - testGetRoleSelectList\_Success() 10ms
  - SysRoleControllerTest 295ms
    - testAdd\_Success() 244ms
    - testGetList\_WithRoleName() 43ms
    - testSelectList\_ReturnsFormattedDropDown() 8.0ms
- com.group17.web.sys\_user.controller 3.5s
  - SysUserControllerIntegrationTest 3.2s
    - testGetInfo\_shouldReturnUserInfoAndPermissions() 51ms
    - testImageCode\_shouldReturnBase64Captcha() 1.4s
    - testRegister\_shouldCreateNewUser() 1.8s
  - SysUserControllerTest 280ms
    - testRegister\_Success() 11ms
    - testRegister\_UserAlreadyExists() 9.0ms
    - testRegister\_SaveUserFails() 248ms
    - testRegister\_SaveUserRoleFails() 12ms

### Appendix 11 (1): Schematic diagram of some key data tables of Navicat--meeting\_book

| id<br># bigint | room_id<br># int | dept_id<br># int | user_id<br># int | start_time<br>datetime | end_time<br>datetime | status<br>varchar | create_time<br>datetime | use_status<br>varchar(64) | use_desc<br>varchar(255)        | apply_desc<br>varchar(255) | process_user<br>varchar(255) | meeting_name<br>varchar(255) |
|----------------|------------------|------------------|------------------|------------------------|----------------------|-------------------|-------------------------|---------------------------|---------------------------------|----------------------------|------------------------------|------------------------------|
| 17             | 1                | 4                | 14               | 2025-03-18 23:00:00    | 2025-03-19 00:00:00  | 3                 | 2025-03-27 17:46:33     | 4                         | Group project discussion        | Approved                   | Administrator                | Startup Brainstorming        |
| 19             | 5                | 4                | 14               | 2025-03-27 22:00:00    | 2025-03-28 00:00:00  | 1                 | 2025-03-27 17:50:46     | 1                         | Startup competition preparation | Approved                   | Conference Coordinator 1     | Business Pitch Rehearsal     |
| 20             | 2                | 4                | 10               | 2025-03-06 04:00:00    | 2025-03-06 06:00:00  | 1                 | 2025-03-27 19:59:11     | 1                         | Group 17 Weekly Discussion      | Approved                   | Conference Coordinator 1     | Group Assignment Discussion  |
| 21             | 3                | 4                | 10               | 2025-03-13 04:00:00    | 2025-03-13 06:00:00  | 1                 | 2025-03-27 20:01:35     | 1                         | Group 17 Weekly Discussion      | Approved                   | Conference Coordinator 1     | Group Assignment Discussion  |
| 22             | 4                | 1                | 18               | 2025-03-26 22:00:00    | 2025-03-27 00:00:00  | 1                 | 2025-03-27 21:01:16     | 1                         | Extracurricular math learning   | Approved                   | Conference Coordinator 2     | Math Club Workshop 1         |
| 23             | 4                | 6                | 18               | 2025-04-02 22:00:00    | 2025-04-03 00:00:00  | 1                 | 2025-03-27 21:02:40     | 1                         | Extracurricular math learning   | Approved                   | Conference Coordinator 2     | Math Club Workshop 2         |
| 24             | 6                | 9                | 16               | 2025-04-22 03:00:00    | 2025-04-22 06:00:00  | 0                 | 2025-04-21 21:12:29     | 0                         | Self-directed learning          | (Null)                     | (Null)                       | Self-directed learning       |
| 25             | 10               | 9                | 16               | 2025-04-22 23:00:00    | 2025-04-22 01:00:00  | 0                 | 2025-04-21 21:13:46     | 0                         | Self-directed learning          | (Null)                     | (Null)                       | Self-directed learning       |
| 26             | 8                | 9                | 21               | 2025-03-13 18:00:00    | 2025-03-13 19:00:00  | 2                 | 2025-03-28 00:52:18     | 2                         | Student union activity          | Rejected                   | Conference Coordinator 1     | Club Meeting                 |
| 29             | 5                | 2                | 12               | 2025-04-01 08:00:00    | 2025-04-02 08:00:00  | 1                 | 2025-04-17 23:53:00     | 1                         | For discussion                  | agree                      | Conference Coordinator 1     | CP202 Discussion             |
| 30             | 6                | 9                | 16               | 2025-04-13 03:00:00    | 2025-04-13 06:00:00  | 0                 | 2025-04-14 21:12:29     | 0                         | Self-directed learning          |                            |                              | Self-directed learning       |
| 31             | 6                | 9                | 16               | 2025-04-22 03:00:00    | 2025-04-22 06:00:00  | 0                 | 2025-04-21 03:00:00     | 0                         | Self-directed learning          |                            |                              | Self-directed learning       |
| 32             | 6                | 9                | 16               | 2025-04-22 03:00:00    | 2025-04-22 06:00:00  | 0                 | 2025-04-21 03:00:00     | 0                         | Self-directed learning          |                            |                              | Self-directed learning       |
| 33             | 6                | 9                | 16               | 2025-04-22 03:00:00    | 2025-04-22 06:00:00  | 0                 | 2025-04-21 03:00:00     | 0                         | Self-directed learning          |                            |                              | Self-directed learning       |
| 34             | 6                | 9                | 16               | 2025-04-22 03:00:00    | 2025-04-22 06:00:00  | 0                 | 2025-04-21 03:00:00     | 0                         | Self-directed learning          |                            |                              | Self-directed learning       |
| 35             | 6                | 9                | 16               | 2025-04-22 03:00:00    | 2025-04-22 06:00:00  | 0                 | 2025-04-21 03:00:00     | 0                         | Self-directed learning          |                            |                              | Self-directed learning       |
| 36             | 6                | 9                | 16               | 2025-04-22 03:00:00    | 2025-04-22 06:00:00  | 0                 | 2025-04-21 03:00:00     | 0                         | Self-directed learning          |                            |                              | Self-directed learning       |
| 37             | 6                | 9                | 16               | 2025-04-22 03:00:00    | 2025-04-22 06:00:00  | 0                 | 2025-04-21 03:00:00     | 0                         | Self-directed learning          |                            |                              | Self-directed learning       |
| 38             | 6                | 9                | 16               | 2025-04-22 03:00:00    | 2025-04-22 06:00:00  | 0                 | 2025-04-21 03:00:00     | 0                         | Self-directed learning          |                            |                              | Self-directed learning       |
| 39             | 6                | 9                | 16               | 2025-04-22 03:00:00    | 2025-04-22 06:00:00  | 0                 | 2025-04-21 03:00:00     | 0                         | Self-directed learning          |                            |                              | Self-directed learning       |
| 40             | 6                | 9                | 16               | 2025-04-22 03:00:00    | 2025-04-22 06:00:00  | 0                 | 2025-04-21 03:00:00     | 0                         | Self-directed learning          |                            |                              | Self-directed learning       |
| 41             | 1                | 4                | 13               | 2025-04-18 23:00:00    | 2025-04-19 00:00:00  | 0                 | 2025-04-19 02:16:13     | 0                         | Group Discussion                | (Null)                     | (Null)                       | Group Discussion             |
| 43             | 1                | 4                | 13               | 2025-04-21 22:00:00    | 2025-04-21 23:00:00  | 1                 | 2025-04-20 20:31:54     | 1                         | Test1                           | Agree                      | Administrator                | Test1                        |
| 44             | 1                | 4                | 11               | 2025-04-23 22:00:00    | 2025-04-24 06:00:00  | 0                 | 2025-04-20 21:15:01     | 0                         | Test2                           | (Null)                     | Conference Coordinator 1     | Test2                        |
| 46             | 1                | 4                | 13               | 2025-04-22 22:00:00    | 2025-04-23 01:00:00  | 1                 | 2025-04-21 17:37:02     | 1                         | test                            | ok                         | Administrator                | test                         |
| 47             | 1                | 4                | 44               | 2025-04-24 22:00:00    | 2025-04-25 01:00:00  | 1                 | 2025-04-23 23:01:26     | 1                         | 1111                            | 1111                       | Administrator                | 111                          |

### Appendix 11 (2): Schematic diagram of some key data tables of Navicat--meeting\_category

| id<br># bigint | category_name<br>varchar(255) | order_num<br>varchar(255) |
|----------------|-------------------------------|---------------------------|
| 1              | Standard Meeting Room         | 1                         |
| 2              | Multimedia Meeting Ro         | 2                         |
| 4              | Group Discussion Room         | 3                         |
| 5              | Silent Cabin                  | 4                         |
| 6              | Group Discussion Area         | 5                         |
| 7              | Lecture Hall                  | 6                         |

### Appendix 11 (3): Schematic diagram of some key data tables of Navicat--meeting\_room

| id<br># bigint | name<br>varchar(255) | area<br># int | category_id<br># int | address<br>varchar(255)              | contain<br># int | device<br>text                  | image<br>varchar(255)            | status<br>tinyint |
|----------------|----------------------|---------------|----------------------|--------------------------------------|------------------|---------------------------------|----------------------------------|-------------------|
| 1              | Thought Pavilion     | 50            | 4                    | SIP-North Campus-SD373               | 12               | - Smart collaborative display / | http://47.100.1.155:8089/images/ | 1                 |
| 2              | Wisdom Chamber       | 20            | 6                    | SIP-North Campus-Area 1 (SE) on FB5F | 10               | - Mobile group tables and che   | http://47.100.1.155:8089/images/ | 1                 |
| 3              | Inspiration Studio   | 20            | 6                    | SIP-North Campus-Area 2 (NE) on FB5F | 10               | - Mobile group tables and che   | http://47.100.1.155:8089/images/ | 1                 |
| 4              | Enlightenment Hall   | 100           | 7                    | SIP-North Campus-MA105               | 100              | - HD projector or LED display-  | http://47.100.1.155:8089/images/ | 1                 |
| 5              | Knowledge Hub        | 100           | 4                    | SIP-South Campus-BS4114              | 50               | - Smart collaborative display / | http://47.100.1.155:8089/images/ | 1                 |
| 6              | Discernment Study    | 10            | 5                    | SIP-North Campus-FB145               | 6                | - Soundproof walls or booth d   | http://47.100.1.155:8089/images/ | 1                 |
| 7              | Diligent Study       | 120           | 2                    | SIP-North Campus-FB251               | 100              | - HD projector / LED video wa   | http://47.100.1.155:8089/images/ | 1                 |
| 8              | Philosophy Room      | 100           | 1                    | SIP-South Campus-HSG26               | 50               | - Large monitor / basic projec  | http://47.100.1.155:8089/images/ | 1                 |
| 10             | Logic Lounge         | 10            | 5                    | SIP-North Campus-FB147               | 6                | - Soundproof walls or booth d   | http://47.100.1.155:8089/images/ | 1                 |
| 11             | Harmony Chamber      | 150           | 7                    | SIP-South Campus-IRG16               | 200              | - HD projector or LED display-  | http://47.100.1.155:8089/images/ | 1                 |
| 12             | Test                 | 100           | 7                    | SIP-SA                               | 200              | Test                            | http://47.100.1.155:8089/images/ | 1                 |

**Appendix 11 (4): Schematic diagram of some key data tables of Navicat--repair**

| id<br># bigint | content_text<br>B_C longtext | user_id<br># int | create_time<br>⌚ datetime | image<br>B_C varchar(255)        | room_id<br># int | status<br>B_C varchar(2) |
|----------------|------------------------------|------------------|---------------------------|----------------------------------|------------------|--------------------------|
| 4              | Projector Not Working        | 1                | 2025-02-17 08:46:10       | http://47.100.1.155:8089/images/ | 1                | 1                        |
| 5              | Speakers No Sound            | 1                | 2025-02-17 13:18:19       | http://47.100.1.155:8089/images/ | 2                | 0                        |
| 6              | Screen Broken                | 1                | 2025-02-18 17:00:17       | http://47.100.1.155:8089/images/ | 6                | 1                        |

**Appendix 11 (5): Schematic diagram of some key data tables of Navicat--sys\_dept**

| id<br># bigint | dept_name<br>B_C varchar(64)             | order_num<br># int |
|----------------|------------------------------------------|--------------------|
| 1              | School of Science                        | 7                  |
| 2              | Design School                            | 2                  |
| 3              | School of Humanities and Social Sciences | 3                  |
| 4              | International Business School Suzhou     | 4                  |
| 5              | School of Languages                      | 5                  |
| 6              | School of Mathematics and Physics        | 6                  |
| 9              | School of Advanced Technology            | 1                  |
| 10             | Chinese Cultural Teaching Centre         | 8                  |
| 11             | Physical Education Centre                | 9                  |
| 12             | Information Technology Management        | 10                 |

**Appendix 11 (6): Schematic diagram of some key data tables of Navicat--sys\_menu**

| menu_id<br># int | parent_id<br># int | title<br>B_C varchar(36) | code<br>B_C varchar(36) | name<br>B_C varchar(36) | path<br>B_C varchar(36) | url<br>B_C varchar(128) | type<br>B_C varchar(2) | icon<br>B_C varchar(36) | parent_name<br>B_C varchar(36) | order_num<br># int |
|------------------|--------------------|--------------------------|-------------------------|-------------------------|-------------------------|-------------------------|------------------------|-------------------------|--------------------------------|--------------------|
| 2                | 0                  | User Management          | sysuser:index           | userList                | /userList               | views/system/User/User  | 1                      | User                    | Top-level Menu                 | 7                  |
| 3                | 0                  | Role Management          | sysrole:index           | roleList                | /roleList               | views/system/Role/Role  | 1                      | Key                     | Top-level Menu                 | 8                  |
| 4                | 0                  | Menu Management          | sysmenu:index           | menuList                | /menuList               | views/system/Menu/Menu  | 1                      | Menu                    | Top-level Menu                 | 9                  |
| 8                | 2                  | Add                      | sysuser:add             |                         |                         |                         | 2                      |                         | User Management                | 21                 |
| 11               | 2                  | Edit                     | sysuser:edit            |                         |                         |                         | 2                      |                         | User Management                | 22                 |
| 12               | 2                  | Delete                   | sysuser:delete          |                         |                         |                         | 2                      |                         | User Management                | 23                 |
| 13               | 3                  | Add                      | sysrole:add             |                         |                         |                         | 2                      |                         | Role Management                | 31                 |
| 14               | 3                  | Edit                     | sysrole:edit            |                         |                         |                         | 2                      |                         | Role Management                | 32                 |
| 15               | 3                  | Delete                   | sysrole:delete          |                         |                         |                         | 2                      |                         | Role Management                | 33                 |
| 16               | 4                  | Add                      | sysmenu:add             |                         |                         |                         | 2                      |                         | Menu Management                | 41                 |
| 17               | 4                  | Edit                     | sysmenu:edit            |                         |                         |                         | 2                      |                         | Menu Management                | 42                 |
| 18               | 4                  | Delete                   | sysmenu:delete          |                         |                         |                         | 2                      |                         | Menu Management                | 43                 |
| 19               | 3                  | Assign Permissions       | sysrole:assign          |                         |                         |                         | 2                      |                         | Role Management                | 34                 |
| 20               | 2                  | Reset Password           | sysuser:reset           |                         |                         |                         | 2                      |                         | User Management                | 24                 |
| 23               | 2                  | Disable User             | sysuser:enabled         |                         |                         |                         | 2                      |                         | User Management                | 25                 |
| 26               | 0                  | Department Management    | sysdept:list            | dept                    | /dept                   | views/sys_dept/index.vu | 1                      | School                  | Top-level Menu                 | 10                 |

Appendix 11 (7): Schematic diagram of some key data tables of Navicat--sys\_role

| role_id<br># int | role_name<br>#C varchar(36)  | type<br>#C varchar(4) | remark<br>#C varchar(64)                            | create_time<br>datetime | update_time<br>datetime |
|------------------|------------------------------|-----------------------|-----------------------------------------------------|-------------------------|-------------------------|
| 13               | System Administrator         | (Null)                | Full access to system settings and user management. | 2025-03-10 08:00:00     | 2025-03-23 20:33:00     |
| 14               | Faculty Member               | (Null)                | Teaching staff with meeting management permissions. | 2025-02-17 08:00:00     | 2025-03-23 20:33:27     |
| 16               | Student                      | (Null)                | Can book meeting rooms and view notices.            | 2025-02-17 08:00:00     | 2025-03-23 20:33:34     |
| 18               | Repair Staff                 | (Null)                | Responsible for maintenance tasks.                  | 2025-02-18 08:00:00     | 2025-03-23 20:33:47     |
| 19               | Visitor                      | (Null)                | Limited viewing access for external guests.         | 2025-03-23 20:34:32     | (Null)                  |
| 20               | Information Technology Staff | (Null)                | Responsible for IT tasks.                           | 2025-03-27 08:00:00     | 2025-03-27 21:26:13     |

Appendix 11 (8): Schematic diagram of some key data tables of Navicat--sys\_user

| user_id<br># int | dept_id<br># int | username<br>#C varchar(100) | password<br>#C varchar(128) | phone<br>#C varchar(18) | email<br>#C varchar(50) | sex<br>#C varchar(4) | is_admin<br>#C varchar(2) |
|------------------|------------------|-----------------------------|-----------------------------|-------------------------|-------------------------|----------------------|---------------------------|
| 1                | 3                | admin                       | \$2a\$10\$K/oZ8.qNgU.7f     | *****                   | *****                   | 0                    | 1                         |
| 6                | 4                | 2255403                     | \$2a\$10\$WYt7yYQArkZI      | 123456789               | Xinyuan.Zhang2202@st    | 1                    | 0                         |
| 8                | 4                | 2255986                     | \$2a\$10\$7jYUv.fxqgRVM     | 123456789               | Wang2202@student.xjt    | 1                    | 0                         |
| 9                | 4                | 2251997                     | \$2a\$10\$vbNgzAKTv4rk      | 123456789               | Xinrong.Shi22@student   | 1                    | 0                         |
| 10               | 4                | 2251903                     | \$2a\$10\$QcbgDLAEfwi       | 123456789               | Hanqiu.Zhang2202@sti    | 1                    | 0                         |
| 11               | 4                | 2255207                     | \$2a\$10\$yMyJm1NFzgx       | 123456789               | Xiaoya.Li22@student.xj  | 1                    | 0                         |
| 12               | 4                | 2252173                     | \$2a\$10\$RAF2atKQ2DL       | 123456789               | Nan.Lin22@student.xjtl  | 0                    | 0                         |
| 13               | 4                | 2147369                     | \$2a\$10\$A7nBAxZTm.rg      | 123456789               | Zimo.Zheng21@studen     | 0                    | 0                         |
| 14               | 4                | 2251651                     | \$2a\$10\$MJXQciR7Oqse      | 18913545693             | Xiangyi.Zhu22@studeni   | 1                    | 0                         |
| 15               | 4                | 2253073                     | \$2a\$10\$OLeCu9EmYwq       | 123456789               | Ziwei.Zhou22@student.   | 1                    | 0                         |

Appendix 12: Unit Testing-User Registration

```
@Test
void testRegister_Success() {
    // Arrange:Set up test data and mock dependencies
    SysUser newUser = new SysUser();
    newUser.setUserId(1L); // Set user ID
    newUser.setUsername("testUser"); // Set username
    newUser.setPassword("123456"); // Set password

    // Mock service layer responses to simulate scenarios:
    when(sysUserService.getById(any(Long.class))).thenReturn(null); // User does not exist
    when(sysUserService.save(any(SysUser.class))).thenReturn(true); // User saved successfully
    when(sysUserRoleService.save(any(SysUserRole.class))).thenReturn(true); // User-role saved successfully

    // Act:Execute the method under test
    ResultVo result = sysUserController.register(newUser);

    // Assert: Verify the expected outcomes
    assertEquals(200, result.getCode()); // Check status code
    assertEquals("Registration successful!", result.getMsg()); // Verify success message

    // Verify service layer interactions:
    verify(sysUserService, times(1)).getById(any(Long.class)); //User existence check was performed
    verify(sysUserService, times(1)).save(any(SysUser.class)); //User creation was attempted
    verify(sysUserRoleService, times(1)).save(any(SysUserRole.class)); //Role assignment was attempted
}
```

## Appendix 13: Integration Testing-Create new booking

```
@SpringBootTest
@AutoConfigureMockMvc
public class MeetingBookIntegrationTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper objectMapper;

    @Test
    public void testCreateMeetingBook() throws Exception {
        // Arrange: Prepare test data
        MeetingBookRequest request = new MeetingBookRequest();
        request.setTitle("Team Meeting"); // Set meeting title
        request.setRemark("Discuss project updates"); // Add meeting description
        request.setStartTime(LocalDateTime.parse("2025-04-20T10:00:00")); // Define start time
        request.setEndTime(LocalDateTime.parse("2025-04-20T11:00:00")); // Define end time

        // Act & Assert: Execute request and validate response
        mockMvc.perform(post("/api/meeting-book") // POST request to endpoint
            .contentType(MediaType.APPLICATION_JSON) // Set JSON content type
            .content(objectMapper.writeValueAsString(request))) // Serialize request body
            .andExpect(status().isOk()) // Verify 200 OK status
            .andExpect(jsonPath("$", is(not(empty())))); // Check non-empty response
            // .andExpect(jsonPath("$.roomId", notNullValue())) // Ensure "id" exists in response
            // .andExpect(jsonPath("$.title", is("Team Meeting"))) // Check for title
            // .andExpect(jsonPath("$.remark", is("Discuss project updates"))); // Check for remark
    }
}
```

## **Development Tools**

### **Version Control:**

- Git (Code Management)
- Coding.net (Remote Repository)

### **Project Management:**

- Coding.net (Scrum Task Management)
- Wiki (Document Collaboration)

### **Database Management:**

- Navicat (11.1.13) (Visual Database Management)

### **IDE & Editors:**

- Visual Studio Code (1.98.0) (Frontend & Backend Development)

### **Documents & Notes:**

- Word (Official Documentation)

## **Programming Languages**

### **Frontend:**

- HTML, CSS (Basic Page Structure and Styling)
- JavaScript (Frontend Interaction)
- TypeScript (Enhancing Code Robustness)

### **Backend:**

- Java 17 (17.0.14)

## **Main Frameworks & Libraries**

### **Frontend:**

- Vue 3 (3.3.4) (Frontend Framework)
- Vite (4.4.5) (Build Tool)
- Vue Router (4.2.4) (Routing Management)
- Element Plus (2.3.12) (UI Component Library)
- Axios (1.7.2) (Frontend API Requests)
- ECharts (5.4.3) (Charts and Graphs)
- FullCalendar + Plugin Set (6.1.15) (Calendar Component)
- Vue-Tippy (6.6.0) (Tooltip Component)

### **Backend:**

- Spring Boot (3.2.7) (Main Framework)
- Maven (3.8.1) (Dependency Management and Packaging Tool)
- Druid (1.2.1) (Database Connection Pool)
- MyBatis-Plus + MyBatis-Plus-Join (3.5.7 + 1.4.7.2) (ORM Framework)
- Sa-Token (Authentication and Authorization) (1.38.0)
- Fastjson (1.2.68) (JSON Processing Tool)
- Spring Security Crypto, JWT (Java-JWT) (6.1.4 + 3.10.3) (Security Components)

### **Database:**

- MySQL (5.7.27) (Relational Database)

### **Authentication:**

- JWT (JSON Web Token)

### **Testing Frameworks:**

- Jest / Mocha (Frontend & Backend Testing)
- JUnit (Java Unit Testing)

## **Required Knowledge**

### **Software Engineering & Development Process**

- Scrum Agile Development
- Sprint Planning, Iterative Development, Daily Standups
- PBI (Product Backlog Item) Management
- Task Assignment (Epic / Story / Task)

### **Database Design**

- Database Table Design (ERD Diagrams)
- Relational Database
- SQL Query Optimization (Indexes, Transactions)

### **Frontend Technologies**

- Vue 3 Component-Based Development
- Vue Router Page Navigation Control
- Axios Network Request Interception
- ECharts Data Visualization
- Element Plus Form Validation and Dialog Boxes

### **Backend Technologies**

- Spring Boot Controller-Service-Mapper Layered Architecture
- MyBatis-Plus Dynamic Conditional Query Encapsulation
- Sa-Token Login Authentication and Role-Based Access Control
- Email Registration & Verification Code Sending Logic
- Image Uploading and Static Resource Management (Local Path Mapping)

### **Deployment and Delivery**

- Development Environment:
- Local Running + Vite Hot Reload + Maven Build
- Server Deployment:
- Cloud Server (Alibaba Cloud)
- Nginx Reverse Proxy
- Node.js Environment Version: 18.20.8
- Frontend Deployment:
- Deployment to the dist Directory on the Cloud Server
- Backend Deployment:
- Upload and Run Packaged .jar File (Spring Boot Standalone Application)

# **Suggested Workflow**

## **Project Initialization**

- Initialize frontend and backend using Vue + Vite + Spring Boot
- Set up Git project structure: main / develop / student ID branches
- Configure a unified API response format and exception handling mechanism

## **Requirements Analysis & Task Assignment**

- Record requirements using Wiki
- Assign tasks via Coding.net
- Create ER diagrams based on user stories
- Plan functional modules, interface prototypes, and permission/role systems

## **Development Stage**

- Frontend: Interface development, database interactions, validation logic encapsulation
- Backend: UI development, API calls, state management synchronization

## **Testing and Integration**

- Console Debugging, User Behavior Tracking (using Sentry)

## **Deployment**

- Package backend into a .jar file
- Build frontend with Vite and upload to server

## **CPT202 Assignment 2**

### **Individual Contribution for Group Report and Presentation**

**Group Number:17**

| <b>Name</b>      | <b>ID Number</b> | <b>Contribution (%)</b> |
|------------------|------------------|-------------------------|
| 1. Xiangyi Zhu   | 2251651          | 13.6                    |
| 2.Xinrong Shi    | 2251997          | 10.8                    |
| 3.Zimo Zheng     | 2147369          | 10.8                    |
| 4.Nan Lin        | 2252173          | 10.8                    |
| 5.Xiaoya Li      | 2255207          | 10.8                    |
| 6.Ziwei Zhou     | 2253073          | 10.8                    |
| 7. Jiayi Wang    | 2255986          | 10.8                    |
| 8.Hanqiu Zhang   | 2251903          | 10.8                    |
| 9. Xinyuan Zhang | 2255403          | 10.8                    |

**Signed by all members:**